

VPA IGCSE COMPUTER SCIENCE (0478)

— YEAR 10 SEMESTER 1

WEEKS 1–9: SYLLABUS GAP FILLING & CONSOLIDATION

Victoria Park Academy
Shenzhen #

EAL Context | Mixed Ability |
40-min Lessons #

COURSE OVERVIEW

This semester fills critical syllabus gaps NOT covered in Year 9 and begins

consolidation of all IGCSE Computer Science (0478) content for examination

preparation.

Year 9 Recap — Topics Already Covered:

Y9S1:

- Data Representation (binary, hex, sound, images, compression)
- Data Transmission (packet switching, error detection, protocols)
 - Hardware (CPU, memory, I/O, storage)
 - Software (system software, application software)
- Internet & Security (threats, prevention, emerging tech)
 - Brief Python introduction

Y9S2:

- Algorithms, Pseudocode, Flowcharts
- Searching (linear search, binary search)
- Python programming (variables, I/O, operators, selection, iteration,

arrays, strings, functions, file handling)

- Testing strategies

- Development lifecycle

SYLLABUS GAPS FILLED THIS SEMESTER (NEW CONTENT)

Topic 9 — Databases & SQL (NOT covered in Y9)

- 9.1 Flat-file vs relational databases
- 9.2 Tables, records, fields, primary keys
- 9.3 Entity-Relationship Diagrams (ERDs)
- 9.4 Relationships (1:1, 1:M, M:N)
- 9.5 SQL: SELECT, FROM, WHERE, AND, OR
- 9.6 SQL: ORDER BY, wildcards, BETWEEN, IN
- 9.7 Foreign keys, linking tables
- 9.8 JOINS (INNER JOIN)
- 9.9 Normalisation (1NF, 2NF, 3NF)

Topic 10 — Boolean Logic (NOT covered in Y9)

- 10.1 AND, OR, NOT gates — truth tables
- 10.2 Logic expressions and circuit diagrams
- 10.3 NAND, NOR, XOR gates
- 10.4 Truth tables with 3 inputs
- 10.5 Half-adders
- 10.6 NAND as universal gate

Topic 7 — Bubble Sort (sorting NOT covered in Y9)

- 7.1 Bubble sort algorithm step-by-step
- 7.2 Pseudocode and Python implementation
- 7.3 Efficiency analysis (best/average/worst case)

Topic 8 — Validation Checks (NOT covered in Y9)

- 8.1 Range check, type check, length check
- 8.2 Presence check, format check
- 8.3 Combining validation checks
- 8.4 Writing robust input routines

Topic 3.4 — Network Hardware (briefly touched only in Y9)

- 3.4.1 NIC, MAC address, IP address (IPv4/IPv6)
- 3.4.2 Routers, switches
- 3.4.3 Network topologies (star, bus, mesh)
- 3.4.4 Client-server vs peer-to-peer

SEMESTER STRUCTURE

Weeks 1–3: Databases & SQL (Topic 9)

Week 4–5: Boolean Logic (Topic 10)

Week 6: Sorting Algorithms — Bubble Sort (Topic 7)

Week 7: Validation Checks (Topic 8)

Week 8: Network Hardware in Depth (Topic 3.4)

Week 9: Paper 1 Review — Data Representation Consolidation

ASSESSMENT ALIGNMENT

Paper 1 Topics Covered This Semester:

- 1.1 Binary & Hexadecimal (review)
- 1.2 Images, Sound, Compression (review)
- 3.3 Network Hardware (detailed)
- 3.4 MAC/IP addressing
- 9.1–9.9 Databases (all subtopics)
- 10.1–10.6 Boolean Logic (all subtopics)

Paper 2 Topics Covered This Semester:

- 2.1 Algorithms — Bubble Sort
- 2.2 Validation checks
- 2.9 SQL (Cambridge pseudocode style)

DIFFERENTIATION FRAMEWORK

Emerging — Students who need significant scaffolding, sentence starters, bilingual glossaries, and additional teacher support.

Developing — Students working at expected level with standard tasks.

Secure — Students who grasp concepts quickly and need extension challenges, optimisation problems, and independent projects.

EAL SUPPORT STRATEGIES (Used Throughout)

- Bilingual keyword lists (English / Chinese / Pinyin) in every lesson
- Visual anchors: diagrams, colour-coded syntax highlighting
- Sentence frames for explanations
- Think-Pair-Share before whole-class responses
- Cognates highlighted (e.g., database → 数据库)
- Word banks on every worksheet
- Code comments in both languages where appropriate
- Wait time increased to 5+ seconds after questioning
- Real-world examples from Chinese context where possible

SQL CONVENTION (Cambridge IGCSE Standard)

This curriculum uses Cambridge-standard simplified SQL:

```
SELECT field1, field2 FROM table
```

SELECT * FROM table

SELECT * FROM table WHERE condition

SELECT * FROM table WHERE condition1 AND condition2

SELECT * FROM table WHERE condition1 OR condition2

SELECT * FROM table ORDER BY field ASC

SELECT * FROM table ORDER BY field DESC

SELECT * FROM table WHERE field LIKE '%text%'

SELECT * FROM table WHERE field BETWEEN value1 AND value2

SELECT * FROM table WHERE field IN (value1, value2)

JOIN syntax (Cambridge style):

SELECT * FROM table1 INNER JOIN table2 ON table1.key = table2.key

Note: Cambridge uses simplified SQL without full ANSI complexity.

Students learn standard SQLite syntax practised in Thonny.

PYTHON ENVIRONMENT

IDE: Thonny (same as Y9 for consistency)

Python Version: 3.10+

Database: SQLite3 (built into Python, works in Thonny)

All programs saved with descriptive filenames in student folders.

BOOLEAN LOGIC NOTATION

Cambridge IGCSE uses standard gate symbols and this notation:

AND = dot or written as AND

OR = + or written as OR

NOT = bar over variable or written as NOT

NAND = AND followed by NOT

NOR = OR followed by NOT

XOR = exclusive OR (only true when inputs differ)

TRUTH TABLE FORMAT (Cambridge Standard)

Inputs shown on left, outputs on right.

For 2 inputs: 4 rows (00, 01, 10, 11)

For 3 inputs: 8 rows (000 through 111)

WEEK 1: DATABASES — INTRODUCTION

Overview: This week introduces databases — one of the most practical and examined topics in IGCSE Computer Science. Students learn what databases are, why they matter, and how flat-file systems differ from relational databases. They learn the core vocabulary (tables, records, fields, primary keys) and

progress to designing Entity-Relationship Diagrams (ERDs) for real-world scenarios. All examples use culturally relevant contexts: school records, library systems, and online shopping familiar to Shenzhen students.

Syllabus Ref: CIE 0478 Topic 9.1–9.4

LESSON 1.1 — WHAT IS A DATABASE?

Topic: Purpose of Databases, Flat-file vs Relational, Tables/Records/Fields, Primary Keys

Duration: 40 minutes

Lesson Type: Conceptual Introduction with Practical Examples

Syllabus Ref: CIE 0478 Topic 9.1, 9.2

Learning Objective:

By the end of this lesson, all students will be able to define a database, explain the difference between flat-file and relational databases, identify tables, records, and fields, and understand the purpose of a primary key.

Key Vocabulary:

- database = 数据库 (shù jù kù)
- table = 表 (biǎo)
- record = 记录 (jì lù)
- field = 字段 (zì duàn)
- primary key = 主键 (zhǔ jiàn)
- flat-file database = 平面文件数据库 (píng miàn wén jiàn shù jù kù)
- relational database = 关系型数据库 (guān xì xíng shù jù kù)
- data redundancy = 数据冗余 (shù jù rǒng yú)
- data inconsistency = 数据不一致 (shù jù bù yī zhì)
- entity = 实体 (shí tǐ)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | Display on board: "Think of an app or website you use that
- | stores LOTS of information. What information does it store?
- | How is it organised?" Examples: WeChat, Taobao, school app.
- | Students write 2 sentences. Teacher picks 2 to share.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) What is a database?
- | - A structured collection of data stored on a computer
- | - Organised so information can be easily searched,
- | sorted, and retrieved

- | - Real-world examples:
- | - School database — stores all student records
- | - Library database — stores books, borrowers, loans
- | - Taobao database — products, customers, orders
- | - Hospital database — patients, doctors, appointments

| (Slide 2) Database components:

- | TABLE — a collection of related data (like a single spreadsheet)
- | RECORD — one row in a table = one complete set of data
- | FIELD — one column in a table = one type of data

| Example — Student table:

StudentID	Name	Class	Age
S001	Zhang Wei	10A	15
S002	Li Na	10B	16
S003	Wang Jun	10A	15

| FIELDS (columns)

| (Slide 3) PRIMARY KEY:

- | - A field that uniquely identifies each record
- | - No two records can have the same primary key value
- | - Cannot be empty (NULL)
- | - Example: StudentID is a good primary key (every student has a unique ID). Name is NOT a good primary key (two students could have the same name).

| (Slide 4) FLAT-FILE vs RELATIONAL:

| FLAT-FILE database:

- | - All data stored in ONE table
- | - Simple but causes problems
- | - Problems:
 - | - Data REDUNDANCY: same data repeated many times
 - | - Data INCONSISTENCY: if you update in one place but forget another, data conflicts
 - | - Wasted storage space

| RELATIONAL database:

- | - Data stored in MULTIPLE linked tables
- | - Tables connected by relationships
- | - Eliminates redundancy — each fact stored only once
- | - More efficient, more powerful

12-25 min | GUIDED PRACTICE

| Whole-class analysis: "The Library Problem"

| Show a flat-file library table with redundancy:

+-----+-----+-----+-----+-----+					
BookID	Title	Author	Borrower	Class	
+-----+-----+-----+-----+-----+					
B001	Physics	Mr Chen	Zhang Wei	10A	
B002	Chemistry	Ms Wang	Zhang Wei	10A	
B003	Biology	Mr Liu	Li Na	10B	
B004	Physics	Mr Chen	Li Na	10B	
+-----+-----+-----+-----+-----+					

| Ask: "What problems do you see?"

- | - Zhang Wei's class repeated twice (redundancy)
- | - If Zhang Wei changes class, must update two records
- | (inconsistency risk)

| - Author "Mr Chen" repeated twice

| Now show the RELATIONAL solution:

| Table: Books

+-----+-----+-----+		
BookID	Title	AuthorID
+-----+-----+-----+		

| Table: Students

+-----+-----+-----+		
StudentID	Name	Class
+-----+-----+-----+		

| Table: Loans

+-----+-----+	
BookID	StudentID
+-----+-----+	

| No redundancy! Each student stored once. Each book stored once.

25-35 min | INDEPENDENT PRACTICE

| Worksheet 1A — Database Concepts

- | Section A: Identify tables, records, fields from diagrams
- | Section B: Flat-file problems (spot redundancy)
- | Section C: Choose a primary key for given scenarios
- | (Worksheet 1A distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Database vocabulary challenge"
- | Teacher says Chinese, students say English:
- | "数据库" -> "database", "主键" -> "primary key", etc.
- | Thumbs up/middle/down: "How confident are you with flat-file vs relational?"
- | Homework: List 3 apps or websites you use that must have a database. For each, name at least 2 tables that probably exist in that database.

EAL Support:

- Pre-printed bilingual word bank on desk (database/table/record/field/primary key)
- Visual diagram of table structure with colour-coding (blue=field names, yellow=records, green=primary key highlighted)
- Sentence frame: "A _____ is a _____ because _____."
- Partner EAL students with bilingual peers for guided practice discussion
- Cognate highlight: "data" -> 数据, "record" -> 记录, "table" -> 表

Differentiation:

- Emerging** | Provide partially labelled table diagrams. Fill-in-the-blank
- | for flat-file vs relational definitions. Give examples from
 - | students' daily life (WeChat contacts as a database).
 - | Complete only Section A of worksheet.

- Developing** | Complete all worksheet sections independently. Identify
- | redundancy in given flat-file tables. Explain in own words
 - | why primary keys are needed.

- Secure** | Extension: Design a flat-file table for a cinema booking
- | system, then explain THREE specific problems caused by
 - | redundancy. Suggest how splitting into relational tables
 - | would solve each problem.

Resources Needed:

- Projector with slides (table diagrams, flat-file vs relational)
- Printed table diagram handouts (A4, colour)
- Worksheet 1A (printed, one per student)

- Whiteboard markers for student sharing
- Exit ticket slips

LESSON 1.2 — ENTITY-RELATIONSHIP DIAGRAMS (ERDs)

Topic: ERDs, One-to-One, One-to-Many, Many-to-Many Relationships

Duration: 40 minutes

Lesson Type: Visual/Design with Practical Drawing

Syllabus Ref: CIE 0478 Topic 9.3, 9.4

Learning Objective:

By the end of this lesson, all students will be able to draw an ERD showing entities and relationships, identify one-to-one, one-to-many, and many-to-many relationships, and design an ERD for a given scenario.

Key Vocabulary:

- Entity-Relationship Diagram (ERD) = 实体关系图 (shí tǐ guān xì tú)
- entity = 实体 (shí tǐ)
- relationship = 关系 (guān xì)
- one-to-one = 一对一 (yī duì yī)
- one-to-many = 一对多 (yī duì duō)
- many-to-many = 多对多 (duō duì duō)
- cardinality = 基数 (jī shù)
- link table / junction table = 连接表 (lián jiē biǎo)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Match the relationship" — Display 3 scenarios:
 - | 1. A country has one capital city
 - | 2. A teacher has many students
 - | 3. Students enrol in many courses
- | Students match to: one-to-one / one-to-many / many-to-many
- | Write answers in notebook. Check with partner.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) What is an ERD?
 - | - A diagram showing the entities in a database and how they relate to each other
 - | - Helps plan a relational database BEFORE building it
 - | - Saves time and prevents mistakes
- | (Slide 2) ERD Symbols (Cambridge standard):

| +-----+

| | ENTITY | Rectangle = an entity (a table)

| | (Student) |

| +-----+

| -----<> Line with diamond = relationship

| 1 ----- M One-to-many (M = many)

| M ----- N Many-to-many

| (Slide 3) Relationship types explained:

| ONE-TO-ONE (1:1):

| Each record in Table A matches exactly ONE record in Table B.

| Example: Student — StudentLocker

| One student has one locker. One locker belongs to one student.

| ONE-TO-MANY (1:M):

| One record in Table A matches many records in Table B.

| Example: Teacher — Student

| One teacher teaches many students. Each student has one form teacher.

| MANY-TO-MANY (M:N):

| Many records in Table A match many records in Table B.

| Example: Student — Course

| One student takes many courses. One course has many students.

| REQUIRES a link/junction table to implement properly.

| (Slide 4) Worked example — School Database ERD:

| +-----+ 1:M +-----+

| | Teacher |-----| Student |

| +-----+ +-----+

| +-----+ M:N +-----+

| | Student |-----| Course |

| +-----+ +-----+

| (needs link table)

| How to read: "One Teacher has many Students."

| "Many Students take many Courses."

12-25 min | GUIDED PRACTICE

| Whole-class: Design an ERD for "Online Shop"

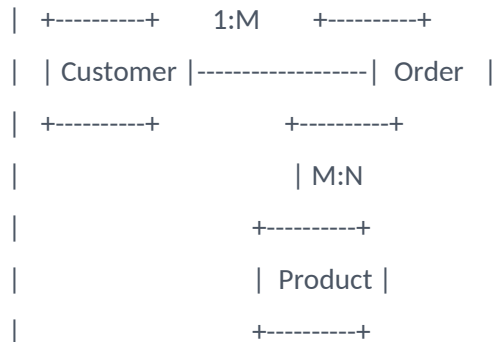
| Step 1: Identify entities (tables)

| - Customer, Product, Order

| Step 2: Identify relationships

- | - Customer places Order -> 1:M (one customer, many orders)
- | - Order contains Product -> M:N (one order has many products;
| one product can be in many orders)

| Step 3: Draw the ERD together on board



| Step 4: Add link table for M:N

| Order_Product table needed: OrderID + ProductID

| Second example (pairs): Hospital database

| Entities: Patient, Doctor, Appointment

| Patient has Appointment with Doctor

| Patient -> Appointment: 1:M

| Doctor -> Appointment: 1:M

| Draw ERD in pairs, one pair shares.

25-35 min | **INDEPENDENT PRACTICE**

| Worksheet 1B — ERD Design Tasks

| Section A: Identify relationship type from description (5 questions)

| Section B: Draw ERDs for 3 scenarios (school, library, online shop)

| Section C: Explain why many-to-many needs a link table

| (Worksheet 1B distributed — see below)

35-40 min | **PLENARY + HOMEWORK**

| Plenary: "ERD Speed Round" — Teacher describes a scenario,

| students hold up fingers: 1 for 1:1, 2 for 1:M, 3 for M:N

| Scenarios:

| - Person -> Passport (1)

| - Class -> Student (2)

| - Actor -> Movie (3)

| - Country -> President (1)

| - Author -> Book (2)

| Homework: Draw a complete ERD for VPA Shenzhen's

| extracurricular activities system. Entities needed:

- | Student, Activity, Teacher. Identify all relationships
- | with correct cardinality. Add a link table if needed.

EAL Support:

- Pre-printed diagram of ERD symbols with labels in both languages
- Sentence frame: "One _____ has many _____."
- Colour-code relationships: blue=1:1, green=1:M, orange=M:N
- Visual scaffold: M:N always needs a "bridge" table (draw as bridge)
- Partner discussion before writing answers

Differentiation:

- Emerging | Provide pre-drawn entity boxes — students only draw lines
- | and label relationship numbers. Use matching activity:
 - | match scenario to relationship type. Complete only Section A.

- Developing | Complete all sections independently. Draw ERDs from
- | written scenarios. Explain relationship choices in
 - | sentences.

- Secure | Extension: Design an ERD for a Shenzhen Metro system.
- | Entities: Station, Line, Passenger (with smart card).
 - | Include: Station->Line (M:N, a station is on multiple lines),
 - | Passenger->Journey (1:M). Add all attributes and
 - | identify primary keys for each entity.

Resources Needed:

- Slide deck (ERD symbols, worked examples)
- Rulers for drawing straight lines
- Worksheet 1B (one per student)
- Coloured pencils for relationship coding
- Whiteboard for modelling ERDs

WORKSHEET 1A — DATABASE CONCEPTS: TABLES, RECORDS, FIELDS & KEYS

Name: _____ Class: _____ Date: _____

Learning Objective: I can identify tables, records, and fields in a database, explain the difference between flat-file and relational databases, and understand primary keys.

Bilingual Support:

- database = 数据库 (shù jù kù)
- table = 表 (biǎo)
- record = 记录 (jì lù)

- field = 字段 (zì duàn)
- primary key = 主键 (zhǔ jiàn)
- flat-file = 平面文件 (píng miàn wén jiàn)
- relational = 关系型 (guān xì xíng)
- redundancy = 冗余 (rǒng yú)
- entity = 实体 (shí tǐ)

SECTION A: Identify Database Components [6 marks]

Study this table:

StudentID	Name	Class	Score
S101	Chen Hua	10A	85
S102	Liu Mei	10B	92
S103	Zhang Li	10A	78
S104	Wang Tao	10C	88

1. How many records are in this table? _____ [1 mark]
2. How many fields are in this table? _____ [1 mark]

3. List the names of all fields:

_____ [1 mark]

3. What is the primary key in this table? Circle it: [1 mark]

StudentID Name Class Score

5. Why is Name NOT a good choice for primary key?

_____ [1 mark]

6. Why is StudentID a good choice for primary key?

_____ [1 mark]

SECTION B: Flat-file vs Relational [8 marks]

4. A school stores all student information in ONE large spreadsheet.
Each row includes: StudentID, Name, Class, TeacherName, TeacherRoom, Subject1, Subject2, Subject3.
 - (a) Explain what data redundancy means in this context. [2 marks]
 - (b) Give ONE specific example of redundancy from this spreadsheet. [1 mark]
 - (c) Explain what data inconsistency means and give an example. [2 marks]
 - (d) Explain how a relational database would solve these problems. [3 marks]

SECTION C: Choosing Primary Keys [6 marks]

For each scenario, suggest a good primary key and explain why.

8. A table storing books in a school library.

Primary key: _____ [1]

Explanation: _____ [1]

9. A table storing orders from an online shop.

Primary key: _____ [1]

Explanation: _____ [1]

10. A table storing countries of the world.

Primary key: _____ [1]

Explanation: _____ [1]

WORKSHEET 1B — ENTITY-RELATIONSHIP DIAGRAMS (ERDs)

Name: _____ Class: _____ Date: _____

Learning Objective: I can draw ERDs showing entities, relationships, and cardinality for real-world scenarios.

Bilingual Support:

- ERD = 实体关系图 (shí tǐ guān xì tú)
- entity = 实体 (shí tǐ)
- relationship = 关系 (guān xì)
- one-to-one = 一对一 (yī duì yī)
- one-to-many = 一对多 (yī duì duō)
- many-to-many = 多对多 (duō duì duō)
- cardinality = 基数 (jī shù)
- link table = 连接表 (lián jiē biǎo)

SECTION A: Identify the Relationship Type [5 marks]

Write 1:1, 1:M, or M:N for each scenario.

1. A person has one passport. _____ [1]

2. A teacher teaches many students. _____ [1]

5. A student takes many courses; a course has many students. _____ [1]

4. A car has one registration number. _____ [1]

5. A hospital ward has many patients. _____ [1]

SECTION B: Draw Entity-Relationship Diagrams [9 marks]

For each scenario, draw an ERD showing entities, relationships, and cardinality.

Use rectangles for entities and lines with 1/M/N labels.

6. School System:

Entities: Teacher, Class

One teacher teaches many classes. Each class has one teacher.

[Draw your ERD here]

[6 marks — 2 for correct entities, 2 for correct relationship, 2 for labels]

7. Library System:

Entities: Book, Author, Borrower

One author writes many books. Each book has one author.

Many borrowers can borrow many books.

[Draw your ERD here]

[6 marks — 2 for correct entities, 2 for relationships, 2 for labels]

8. Online Shopping System:

Entities: Customer, Product, Order

One customer places many orders. Each order belongs to one customer.

Each order contains many products. Each product can be in many orders.

[Draw your ERD here]

[6 marks — 2 for correct entities, 2 for relationships, 2 for labels
and link table]

SECTION C: Explain Many-to-Many [6 marks]

6. Explain why a many-to-many relationship needs a link table. Use an example in your answer. [4 marks]

7. What fields would the link table contain for a Student-Course M:N relationship? [2 marks]

ANSWER KEY — WEEK 1

WORKSHEET 1A:

SECTION A:

8. 4 records [1]

9. 4 fields [1]

3. StudentID, Name, Class, Score [1] (any order)

4. StudentID circled [1]

10. Because two students could have the same name [1]. The primary key must be unique for every record [1]. Accept equivalent explanation.

11. Because every student is given a unique StudentID [1]. No two students will ever have the same ID, so it can always identify exactly one record [1].

SECTION B:

12. (a) Data redundancy means the same data is stored more than once [1].

In this spreadsheet, the teacher's name and room are repeated for every student in that class [1].

(b) Any valid example, e.g., "TeacherName 'Mr Zhang' is repeated for every student in class 10A" [1]. Accept any specific example.

(c) Data inconsistency means the same data has different values in different places [1]. Example: if Mr Zhang moves to a different room, someone might update the room number for some students but not others, so some records show the old room and some show the new room [1].

(d) Split the data into separate tables: one table for Students (StudentID, Name, Class), one table for Teachers (TeacherName, TeacherRoom), and one table linking them [1]. Each teacher's information is stored only once [1], so there is no redundancy, and updating the teacher's room only requires changing one record [1].

SECTION C:

13. ISBN or BookID [1]. Every book has a unique ISBN number, so it can always identify exactly one book [1]. Accept "BookID" with valid explanation.

14. OrderID [1]. Every order is given a unique order number, so it can always identify exactly one order [1]. Accept "OrderNumber".

15. CountryCode or CountryName [1]. Each country has a unique code (e.g., "CN" for China) that identifies only that country [1]. Accept "CountryName" with explanation.

WORKSHEET 1B:

SECTION A:

16. 1:1 [1]
17. 1:M [1]

3. M:N [1]

18. 1:1 [1]
19. 1:M [1]

SECTION B:

20. Teacher ---1:M--- Class [2 for correct entities, 2 for 1:M relationship, 2 for correct labels]
21. Author ---1:M--- Book and Book ---M:N--- Borrower [2 for correct entities, 2 for relationships (1:M and M:N), 2 for correct labels and link table for Book-Borrower]

8. Customer ---1:M--- Order and Order ---M:N--- Product

Link table needed: Order_Product [2 for correct entities, 2 for relationships, 2 for link table identification]

SECTION C:

22. A many-to-many relationship cannot be implemented directly in a relational database [1]. A link table breaks the M:N relationship into two 1:M relationships [1]. Example: Student-Course M:N becomes Student-LinkTable (1:M) and Course-LinkTable (1:M) [1]. The link table stores pairs of primary keys from both tables [1]. Accept equivalent explanation with any valid example.
23. StudentID and CourseID [1 each]. Accept "the primary keys from both Student and Course tables".

WEEK 2: SQL — BASICS

Overview: This week introduces SQL (Structured Query Language) — the standard language for interacting with databases. Students learn the fundamental SELECT statement, filtering with WHERE, and combining conditions with AND/OR. Lesson 2 extends to sorting (ORDER BY), pattern matching (wildcards), and range queries (BETWEEN, IN). All practical work uses SQLite3 in Thonny with a pre-built sample database. A syntax reference card is provided for student desks. Cambridge IGCSE uses simplified SQL — this matches the CIE style while ensuring students can write working code.

Syllabus Ref: CIE 0478 Topic 9.5, 9.6

Prerequisites:

- Week 1 content (tables, records, fields, primary keys, relationships)
- Basic Python familiarity from Year 9

LESSON 2.1 — INTRODUCTION TO SQL: SELECT, FROM, WHERE

Topic: SQL Syntax, SELECT, FROM, WHERE, Comparison Operators, AND/OR

Duration: 40 minutes

Lesson Type: Practical Coding with Direct Instruction

Syllabus Ref: CIE 0478 Topic 9.5

Learning Objective:

By the end of this lesson, all students will be able to write SQL queries using SELECT, FROM, and WHERE clauses, use comparison operators (=, <, >, <=, >=, <>), and combine conditions with AND and OR.

Key Vocabulary:

- SQL = 结构化查询语言 (jié gòu huà chá xún yǔ yán)
- query = 查询 (chá xún)
- SELECT = 选择/查询 (xuǎn zé / chá xún)
- FROM = 从 (cóng)
- WHERE = 条件 (tiáo jiàn)
- clause = 子句 (zǐ jù)
- comparison operator = 比较运算符 (bǐ jiào yùn suàn fú)
- AND = 与/并且 (yǔ / bìng qiě)
- OR = 或/或者 (huò / huò zhě)
- result set = 结果集 (jié guǒ jí)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| Display this table on the board:

StudentID	Name	Class	Score
S001	Zhang Wei	10A	85
S002	Li Na	10B	92
S003	Wang Jun	10A	78
S004	Chen Mei	10C	65

| "Without using a computer — how would you find all students

| in Class 10A? How would you find students with Score > 80?"

| Write method in notebook. Think-Pair-Share.

3-12 min | DIRECT INSTRUCTION

| (Slide 1) What is SQL?

- | - Structured Query Language
- | - A language for managing and querying databases
- | - Used by almost ALL databases (MySQL, SQLite, Oracle)
- | - Cambridge IGCSE tests SQL knowledge

| (Slide 2) Basic SQL query structure:

| `SELECT field1, field2` <- what columns to show

| `FROM table_name` <- which table to look in

| Example:

| `SELECT Name, Score`

| `FROM Students`

| -> Shows the Name and Score columns from the Students table

| (Slide 3) The WHERE clause — filtering:

| `SELECT *`

| `FROM Students`

| `WHERE Class = '10A'`

| -> Shows ALL columns (*) only for students in class 10A

| (Slide 4) Comparison operators:

| `=` equal to

| `<` less than

| `>` greater than

| `<=` less than or equal to

| `>=` greater than or equal to

| `<>` not equal to

| Examples:

| `SELECT * FROM Students WHERE Score > 80`

| `SELECT * FROM Students WHERE Class <> '10C'`

| `SELECT * FROM Students WHERE Score >= 90`

| (Slide 5) AND and OR:

| AND — both conditions must be true

| `SELECT * FROM Students WHERE Class = '10A' AND Score > 80`

| -> Only Zhang Wei (10A, 85)

| OR — at least one condition must be true

| `SELECT * FROM Students WHERE Class = '10A' OR Score > 90`

| -> Zhang Wei (10A), Li Na (score 92)

12-25 min | GUIDED PRACTICE

- | Teacher opens Thonny with pre-built students.db file.
- | Students follow along on their computers.
- | Exercise 1 (whole class):
 - | "Show all students in class 10B."
 - | -> `SELECT * FROM Students WHERE Class = '10B'`
- | Exercise 2 (pairs):
 - | "Show all students with score greater than 80."
 - | -> `SELECT * FROM Students WHERE Score > 80`
- | Exercise 3 (individual, then check):
 - | "Show the names and scores of all students NOT in class 10A."
 - | -> `SELECT Name, Score FROM Students WHERE Class <> '10A'`
- | Exercise 4 (whole class):
 - | "Show all students in class 10A who scored more than 80."
 - | -> `SELECT * FROM Students WHERE Class = '10A' AND Score > 80`
- | Exercise 5 (challenge):
 - | "Show all students who are in class 10A OR scored more than 90."
 - | -> `SELECT * FROM Students WHERE Class = '10A' OR Score > 90`
- | Common errors to highlight:
 - | - Missing quotes around text values: `Class = 10A` (WRONG)
 - | - Using `=` instead of `<>` for "not equal"
 - | - Forgetting AND/OR when combining conditions

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 2A — SQL Basics: SELECT, FROM, WHERE
- | Section A: Match SQL query to its description
- | Section B: Write SQL queries for 6 scenarios
- | Section C: Spot and correct errors in SQL queries
- | (Worksheet 2A distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "SQL Syntax Race" — Teacher describes what they want,
- | students write the SQL on mini-whiteboards, hold up together.
- | Round 1: Show all fields from Books table
- | Round 2: Show books where price < 50
- | Round 3: Show books where genre = 'Science' AND price > 30
- | Homework: Write SQL queries for:
 - | 1. Show all students whose score is 90 or above

- | 2. Show the names of all students in class 10C
- | 3. Show all students who are NOT in class 10B
- | 4. Show all students who are in class 10A and scored above 80

EAL Support:

- SQL syntax reference card on every desk (bilingual)
- Sentence frame: "SELECT ____ FROM ____ WHERE ____"
- Colour-code SQL keywords: SELECT (blue), FROM (green), WHERE (orange), AND (purple), OR (red)
- Pre-printed table diagram so students can see field names while writing
- Common errors poster on wall

Differentiation:

Emerging | Provide partially completed queries with blanks to fill in.

- | Use matching activity first (match query to result).
- | SQL keyword word bank available. Complete Section A only.

Developing | Write complete queries from English descriptions.

- | Complete all sections. Use SQLite to test queries.

Secure | Extension: Write queries for complex conditions involving

- | nested AND/OR. Example: "Show students in class 10A with
- | score > 80 OR students in class 10B with score > 90."
- | Add brackets: WHERE (Class='10A' AND Score>80) OR
- | (Class='10B' AND Score>90).

Resources Needed:

- Projector showing Thonny with SQLite
- Pre-built students.db database file (shared on network)
- SQL reference cards (printed, laminated, one per student)
- Worksheet 2A (one per student)
- Mini-whiteboards for plenary
- Common SQL errors poster

LESSON 2.2 — MORE SQL: ORDER BY, WILDCARDS, BETWEEN, IN

Topic: Sorting Results, Pattern Matching, Range Queries, List Queries

Duration: 40 minutes

Lesson Type: Practical Coding with Scenario-Based Tasks

Syllabus Ref: CIE 0478 Topic 9.6

Learning Objective:

By the end of this lesson, all students will be able to sort query results

with ORDER BY, use wildcards (%) for pattern matching, filter with BETWEEN and IN, and write complex multi-clause SQL queries.

Key Vocabulary:

- ORDER BY = 排序 (pái xù)
- ASC = 升序 (shēng xù) — ascending
- DESC = 降序 (jiàng xù) — descending
- wildcard = 通配符 (tōng pèi fú)
- LIKE = 模糊匹配 (mó hū pǐ pèi)
- BETWEEN = 在...之间 (zài...zhī jiān)
- IN = 在...之中 (zài...zhī zhōng)
- pattern = 模式 (mó shì)
- sort = 排序 (pái xù)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You have 1000 products in a database. You want to find the
- | 10 cheapest. How would SQL help?"
- | Write answer in notebook. Think-Pair-Share.
- | (Target answer: sort by price, lowest first, then limit to 10)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) ORDER BY — sorting results:
- | `SELECT * FROM Students ORDER BY Score ASC`
- | -> Shows all students, lowest score first (ascending)
- | `SELECT * FROM Students ORDER BY Score DESC`
- | -> Shows all students, highest score first (descending)
- | `SELECT * FROM Students ORDER BY Class, Score DESC`
- | -> Sorts by class first, then by score within each class
- | (Slide 2) Wildcards — pattern matching with LIKE:
- | % means "any characters, any number of them"
- | _ means "exactly one character"
- | `SELECT * FROM Students WHERE Name LIKE 'Zhang%'`
- | -> Finds Zhang Wei, Zhang Li, Zhang Hua, etc.
- | `SELECT * FROM Students WHERE Name LIKE '%Mei'`
- | -> Finds Chen Mei, Li Mei, Wang Mei, etc.
- | `SELECT * FROM Students WHERE Name LIKE 'Z%g'`
- | -> Finds names starting with Z and ending with g

| (Slide 3) BETWEEN — range queries:
| `SELECT * FROM Students WHERE Score BETWEEN 80 AND 90`
| -> Finds students with score 80-90 inclusive
| (Same as: `Score >= 80 AND Score <= 90`)
| (Slide 4) IN — matching a list:
| `SELECT * FROM Students WHERE Class IN ('10A', '10B')`
| -> Finds students in class 10A OR 10B
| (Same as: `Class = '10A' OR Class = '10B'`)
| (Slide 5) SQL Reference Summary Card:
| `SELECT field1, field2` -- choose columns
| `FROM table` -- choose table
| `WHERE condition` -- filter rows
| `ORDER BY field ASC|DESC` -- sort results
| `WHERE Name LIKE '%text%'` -- pattern matching
| `WHERE field BETWEEN a AND b`
| `WHERE field IN (a, b, c)`

12-25 min | **GUIDED PRACTICE**

| Using the students.db database in Thonny:
| Exercise 1 (whole class):
| "Show all students sorted by name in alphabetical order."
| -> `SELECT * FROM Students ORDER BY Name ASC`
| Exercise 2 (pairs):
| "Find all students whose name starts with 'L'."
| -> `SELECT * FROM Students WHERE Name LIKE 'L%'`
| Exercise 3 (individual):
| "Find all students with scores between 70 and 85."
| -> `SELECT * FROM Students WHERE Score BETWEEN 70 AND 85`
| Exercise 4 (whole class):
| "Show all students in classes 10A or 10B, sorted by score
| highest first."
| -> `SELECT * FROM Students WHERE Class IN ('10A','10B')`
| `ORDER BY Score DESC`
| Exercise 5 (challenge — pairs):
| "Find students whose name contains 'Wei', with score > 80,
| sorted by score descending."
| -> `SELECT * FROM Students WHERE Name LIKE '%Wei%'`

| AND Score > 80 ORDER BY Score DESC

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 2B — Advanced SQL Queries
- | Section A: Write the SQL for 8 described queries
- | Section B: Complex scenario — VPA Shenzhen exam results database
- | Section C: Spot errors and rewrite correct SQL
- | (Worksheet 2B distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Query Construction Challenge"
- | Students receive a scenario and build the query step by step:
 - | "A librarian wants to find all books published between 2015 and 2020, with titles containing 'Science', sorted by title.
 - | Build the query together."
- | Step 1: SELECT * FROM Books
- | Step 2: WHERE Year BETWEEN 2015 AND 2020
- | Step 3: AND Title LIKE '%Science%'
- | Step 4: ORDER BY Title ASC
- | Homework: VPA Shenzhen library database has tables:
 - | Books (BookID, Title, Author, Genre, YearPublished, Available)
 - | Write SQL queries for:
 - | 1. Show all books sorted by year published, newest first
 - | 2. Find all books where the title contains 'Computer'
 - | 3. Find all books published between 2018 and 2023
 - | 4. Find all books in the 'Science' or 'Technology' genres

EAL Support:

- SQL reference card with all keywords (bilingual) on every desk
- Sentence frames for complex queries:
 - "I need to SORT the results, so I add ORDER BY _____"
 - "I need to FIND a PATTERN, so I use LIKE with %_____"
- Visual flowchart showing SQL query structure (SELECT -> FROM -> WHERE -> ORDER BY)
- Common SQL error translation guide
- Highlight that % is like a "wildcard" in card games (万能牌)

Differentiation:

- Emerging | Provide SQL keyword cards to physically arrange into correct
- | order. Use "fill in the blanks" for all queries. Focus on
 - | ORDER BY and LIKE only. Complete Section A questions 1-4 only.

Developing | Write complete queries from descriptions. Use all keywords
| correctly. Complete all sections. Test queries in Thonny.

Secure | Extension: Write a single query to solve a multi-part problem.
| "Find all students whose surname starts with Z, who scored
| between 75 and 95, and are NOT in class 10C. Sort by score
| highest first, showing only Name and Score columns."

Resources Needed:

- Thonny with students.db (shared database file)
- SQL reference cards (laminated)
- Worksheet 2B (one per student)
- Whiteboard for building queries together
- Common SQL errors poster

WORKSHEET 2A — SQL BASICS: SELECT, FROM, WHERE

Name: _____ Class: _____ Date: _____

Learning Objective: I can write SQL queries using SELECT, FROM, WHERE,
comparison operators, AND, and OR.

Bilingual Support:

- SQL = 结构化查询语言 (jié gòu huà chá xún yǔ yán)
- SELECT = 选择 (xuǎn zé)
- FROM = 从 (cóng)
- WHERE = 条件 (tiáo jiàn)
- AND = 并且 (bìng qiě)
- OR = 或者 (huò zhě)
- query = 查询 (chá xún)

Use this table for all questions:

Products Table:

ProductID	ProductName	Category	Price	Stock
P001	Wireless Mouse	Electronics	89	45
P002	USB-C Cable	Electronics	35	120
P003	Notebook A5	Stationery	12	200
P004	Eraser Pack	Stationery	8	150
P005	Laptop Stand	Electronics	156	30
P006	Pen Set Blue	Stationery	15	80
P007	Bluetooth Speaker	Electronics	199	25
P008	Folder A4	Stationery	6	300

SECTION A: Match the Query to the Description [4 marks]

Match each SQL query (left) to what it does (right). Write the letter.

24. SELECT * FROM Products [1] a) Show names of stationery items
25. SELECT ProductName FROM Products [1] b) Show all products under 50 RMB
WHERE Category = 'Stationery'
26. SELECT * FROM Products WHERE Price < 50 [1] c) Show all fields for all products
27. SELECT ProductName, Price FROM Products [1] d) Show only name and price
WHERE Stock > 100

SECTION B: Write the SQL Query [12 marks]

Write the correct SQL query for each task.

5. Show all information about all products.

_____ [2]

6. Show the product name and price of all products.

_____ [2]

7. Show all products in the 'Electronics' category.

_____ [2]

8. Show all products that cost more than 100 RMB.

_____ [2]

28. Show all products that are in 'Electronics' AND cost less than 100 RMB.

_____ [2]

29. Show all products that cost less than 20 RMB OR have stock greater than 100.

_____ [2]

SECTION C: Spot the Errors [9 marks]

Each query has ONE error. Find it and write the corrected query.

11. SELECT * Products WHERE Price > 50

Error: _____ [1]

Corrected: _____ [2]

30. SELECT ProductName, Price FROM Products WHERE Category = Electronics

Error: _____ [1]

Corrected: _____ [2]

13. SELECT * FROM Products WHERE Price > 50 AND < 100

Error: _____ [1]

Corrected: _____ [2]

WORKSHEET 2B — ADVANCED SQL: ORDER BY, WILDCARDS, BETWEEN, IN

Name: _____ Class: _____ Date: _____

Learning Objective: I can use ORDER BY, wildcards (%), BETWEEN, and IN in SQL queries, and combine multiple clauses.

Bilingual Support:

- ORDER BY = 排序 (pái xù)
- ASC = 升序 (shēng xù)
- DESC = 降序 (jiàng xù)
- wildcard = 通配符 (tōng pèi fú)
- LIKE = 模糊匹配 (mó hū pǐ pèi)
- BETWEEN = 在...之间 (zài...zhī jiān)
- IN = 在...之中 (zài...zhī zhōng)

Use this table for questions 1-8:

Students Table:

StudentID	Name	Class	Score	Subject
S001	Zhang Wei	10A	85	CS
S002	Li Na	10B	92	Math
S003	Wang Jun	10A	78	CS
S004	Chen Mei	10C	65	Physics
S005	Liu Tao	10B	88	CS
S006	Zhang Li	10A	91	Math
S007	Huang Min	10C	72	CS
S008	Li Wei	10B	95	Physics

SECTION A: Write the SQL Query [16 marks]

Write the correct SQL query for each task.

31. Show all students sorted by score from lowest to highest.

_____ [2]

2. Show all students sorted by name alphabetically (A-Z).

_____ [2]

32. Show all students in class 10A, sorted by score highest first.

_____ [2]

4. Find all students whose name starts with 'Z'.

_____ [2]

5. Find all students whose name contains 'Li'.

_____ [2]

6. Find all students with scores between 70 and 85.

_____ [2]

7. Find all students studying CS or Math.

_____ [2]

33. Show the name and score of all students in classes 10A or 10B,
with score above 80, sorted by score descending.

_____ [2]

SECTION B: VPA Shenzhen Exam Results Database [8 marks]

The school exam database has this table:

ExamResults (StudentID, Name, Subject, Score, Grade, ExamDate)

34. Find all students who got grade 'A' in Computer Science. [2]

35. Find all exam results from June 2024, sorted by score highest first. [2]

36. Find all students whose name starts with 'H' and scored between 70 and 90. [2]

37. Find all Maths and Physics results where the score is at least 85. [2]

SECTION C: Complex Queries [6 marks]

13. Explain what each part of this query does:

SELECT Name, Score FROM Students

WHERE Class = '10A' AND Score > 80

ORDER BY Score DESC

SELECT: _____ [1]

FROM: _____ [1]

WHERE: _____ [1]

ORDER BY: _____ [1]

38. Write a query to find all CS students whose name contains 'Wei'
or 'Zhang', sorted by name alphabetically. [4]

WORKSHEET 2A:

SECTION A:

39. c [1]

40. a [1]

41. b [1]

42. d [1]

SECTION B:

43. SELECT * FROM Products [2] (1 mark for SELECT *, 1 mark for FROM Products)

6. SELECT ProductName, Price FROM Products [2]

44. SELECT * FROM Products WHERE Category = 'Electronics' [2]

8. SELECT * FROM Products WHERE Price > 100 [2]

45. SELECT * FROM Products WHERE Category = 'Electronics' AND Price < 100 [2]

46. SELECT * FROM Products WHERE Price < 20 OR Stock > 100 [2]

Accept equivalent queries. Deduct 1 if quotes missing around 'Electronics'.

SECTION C:

11. Error: Missing FROM keyword [1]

Corrected: SELECT * FROM Products WHERE Price > 50 [2]

12. Error: Missing quotes around 'Electronics' [1]

Corrected: SELECT ProductName, Price FROM Products

WHERE Category = 'Electronics' [2]

47. Error: Incomplete comparison — should be Price > 50 AND Price < 100 [1]

Corrected: SELECT * FROM Products WHERE Price > 50 AND Price < 100 [2]

(Accept BETWEEN 50 AND 100 as alternative correction)

WORKSHEET 2B:

SECTION A:

1. SELECT * FROM Students ORDER BY Score ASC [2]

2. SELECT * FROM Students ORDER BY Name ASC [2]

48. SELECT * FROM Students WHERE Class = '10A' ORDER BY Score DESC [2]

4. SELECT * FROM Students WHERE Name LIKE 'Z%' [2]

5. SELECT * FROM Students WHERE Name LIKE '%Li%' [2]

6. SELECT * FROM Students WHERE Score BETWEEN 70 AND 85 [2]

(Accept WHERE Score >= 70 AND Score <= 85)

49. SELECT * FROM Students WHERE Subject IN ('CS', 'Math') [2]

(Accept WHERE Subject = 'CS' OR Subject = 'Math')

50. SELECT Name, Score FROM Students WHERE Class IN ('10A', '10B')
AND Score > 80 ORDER BY Score DESC [2]

(Deduct 1 if quotes missing or ASC used instead of DESC)

SECTION B:

51. SELECT * FROM ExamResults WHERE Grade = 'A' AND Subject = 'CS' [2]

(Accept: WHERE Subject = 'Computer Science')

52. SELECT * FROM ExamResults WHERE ExamDate LIKE '%June 2024%'
ORDER BY Score DESC [2]

(Accept: WHERE ExamDate BETWEEN '2024-06-01' AND '2024-06-30')

53. SELECT * FROM ExamResults WHERE Name LIKE 'H%' AND Score BETWEEN 70 AND 90
[2]

54. SELECT * FROM ExamResults WHERE Subject IN ('Maths', 'Physics')
AND Score >= 85 [2]

(Accept 'Math' or 'Mathematics')

SECTION C:

55. SELECT: Choose which columns to display (Name and Score) [1]

FROM: Choose which table to search (Students) [1]

WHERE: Filter only class 10A students with score above 80 [1]

ORDER BY: Sort results by score from highest to lowest [1]

14. SELECT * FROM Students WHERE Subject = 'CS' AND

(Name LIKE '%Wei%' OR Name LIKE '%Zhang%') ORDER BY Name ASC [4]

(1 mark for Subject = 'CS', 1 mark for LIKE conditions,

1 mark for OR, 1 mark for ORDER BY Name ASC)

Accept equivalent structures. Brackets around OR condition preferred

but not required for full marks.

WEEK 3: SQL — ADVANCED & DATABASE DESIGN

Overview: This week moves from writing queries to understanding database structure. Lesson 1 covers foreign keys, linking tables, INNER JOINS, and the three normal forms (1NF, 2NF, 3NF) that ensure databases are well-designed. Lesson 2 is a practical design task where students create a complete database with 3+ linked tables for a real-world scenario. This is the most challenging database week — it bridges SQL knowledge with structural understanding and prepares students for exam questions on database design and normalisation.

Syllabus Ref: CIE 0478 Topic 9.7, 9.8, 9.9

Prerequisites:

- Week 1 (ERDs, relationships, primary keys)
- Week 2 (SELECT, WHERE, ORDER BY)

LESSON 3.1 — FOREIGN KEYS, JOINS & NORMALISATION

Topic: Foreign Keys, INNER JOIN, 1NF, 2NF, 3NF

Duration: 40 minutes

Lesson Type: Conceptual with Practical Design Exercises

Syllabus Ref: CIE 0478 Topic 9.7, 9.8, 9.9

Learning Objective:

By the end of this lesson, all students will be able to explain foreign keys and their purpose, write an INNER JOIN query, identify whether a table is in 1NF, 2NF, or 3NF, and explain the normalisation process.

Key Vocabulary:

- foreign key = 外键 (wài jiàn)
- INNER JOIN = 内连接 (nèi lián jiē)
- normalisation = 规范化 (guī fàn huà)
- first normal form (1NF) = 第一范式 (dì yī fàn shì)
- second normal form (2NF) = 第二范式 (dì èr fàn shì)
- third normal form (3NF) = 第三范式 (dì sān fàn shì)
- repeating group = 重复组 (chóng fù zǔ)
- partial dependency = 部分依赖 (bù fèn yī lài)
- transitive dependency = 传递依赖 (chuán dì yī lài)
- link table = 连接表 (lián jiē biǎo)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

| Display two tables: Customers (CustomerID, Name) and
| Orders (OrderID, CustomerID, Amount).
| "How are these tables linked? What field connects them?"
| Why do we need two separate tables instead of one?"
| Write answer. Think-Pair-Share.

3-12 min | **DIRECT INSTRUCTION**

| (Slide 1) FOREIGN KEYS:
| - A field in one table that refers to the primary key
| in another table
| - Creates the link between tables
| - Example:
| Customer table: CustomerID (PK), Name, Address
| Order table: OrderID (PK), CustomerID (FK), Date
| -> CustomerID in Order table is a FOREIGN KEY
| -> It "points to" the primary key in Customer table
| (Slide 2) INNER JOIN — combining tables:
| An INNER JOIN shows only records that have matching
| values in BOTH tables.
| SELECT Customers.Name, Orders.Amount
| FROM Customers
| INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID
| Result: Shows customer names alongside their order amounts,
| but ONLY for customers who actually have orders.
| (Slide 3) Normalisation — why it matters:
| Normalisation = organising data to reduce redundancy
| and prevent problems when adding, updating, or deleting.
| 1NF: No repeating groups. Each cell contains ONE value.
| 2NF: Must be in 1NF + no partial dependencies.
| Every non-key field depends on the WHOLE primary key.
| 3NF: Must be in 2NF + no transitive dependencies.
| No non-key field depends on another non-key field.
| (Slide 4) 1NF Example:
| NOT 1NF:
| +-----+-----+-----+
| | Student | Subject1 | Subject2 | <- repeating group!
| +-----+-----+-----+

Zhang Wei	CS	Math
-----------	----	------

IN 1NF:

Student	Subject
Zhang Wei	CS
Zhang Wei	Math

<- no repeating groups!

(Slide 5) 2NF Example:

Table: Enrolment(StudentID + CourseID -> Grade, TutorName)

Problem: TutorName depends ONLY on CourseID, not the full composite key (StudentID + CourseID).

Solution: Split into Enrolment(StudentID, CourseID, Grade) and Course(CourseID, TutorName)

(Slide 6) 3NF Example:

Table: Student(StudentID -> Name, Class, TeacherName)

Problem: TeacherName depends on Class, not directly on StudentID (transitive dependency: StudentID -> Class -> TeacherName)

Solution: Split into Student(StudentID, Name, ClassID) and Class(ClassID, ClassName, TeacherName)

12-25 min | GUIDED PRACTICE

Exercise 1 (whole class):

Given two tables: Authors(AuthorID, Name) and Books(BookID, Title, AuthorID).

"Write a query to show book titles with author names."

-> SELECT Books.Title, Authors.Name

-> FROM Books

-> INNER JOIN Authors ON Books.AuthorID = Authors.AuthorID

Exercise 2 (pairs):

"This table is NOT in 1NF. Fix it."

Orders(OrderID, Customer, Items)

O001, Zhang Wei, "Mouse, Keyboard, Monitor"

-> Split into: Orders(OrderID, Customer) and OrderItems(OrderID, Item)

Exercise 3 (individual):

- | "Is this table in 2NF? If not, fix it."
- | Enrolment(StudentID, SubjectID -> Grade, SubjectName)
- | -> No — SubjectName depends only on SubjectID.
- | -> Split: Enrolment(StudentID, SubjectID, Grade) and
- | Subject(SubjectID, SubjectName)

25-35 min | **INDEPENDENT PRACTICE**

- | Worksheet 3A — Foreign Keys, Joins & Normalisation
- | Section A: Identify foreign keys in table diagrams
- | Section B: Write INNER JOIN queries
- | Section C: Analyse tables for normal form and suggest fixes
- | (Worksheet 3A distributed — see below)

35-40 min | **PLENARY + HOMEWORK**

- | Plenary: "Normalisation Quick-fire"
- | Teacher describes a table, students hold up fingers:
- | 1 finger = not 1NF, 2 fingers = not 2NF,
- | 3 fingers = not 3NF, thumbs up = already 3NF
- | Example 1: Student(StudentID -> Name, Phone1, Phone2)
- | -> 1 finger (repeating group — not 1NF)
- | Example 2: Enrolment(StudentID+CourseID -> Grade, CourseName)
- | -> 2 fingers (partial dependency — not 2NF)
- | Example 3: Employee(EmpID -> Name, DeptID, DeptName)
- | -> 3 fingers (transitive dependency — not 3NF)
- | Homework: For the school database scenario, explain which
- | normal form each table is in and why:
- | Table: Student(StudentID, Name, Class, ClassTeacher)
- | Is this in 3NF? Explain. If not, redesign.

EAL Support:

- Foreign key diagram showing "points to" relationship with arrows
- Normalisation flowchart: Is there a repeating group? -> 1NF -> Partial dependency? -> 2NF -> Transitive dependency? -> 3NF
- Sentence frames: "_____ is a foreign key because it references the primary key in the _____ table."
- Colour-coding: PK highlighted in yellow, FK highlighted in orange
- Cognate: "normal" -> 规范 (normalisation)

Differentiation:

Emerging | Focus on identifying foreign keys only. Provide pre-drawn

- | table diagrams — students label the FK. For normalisation,
- | identify only whether a table is in 1NF (repeating groups
- | are visually obvious). Complete Sections A and B only.

Developing | Write JOIN queries and identify normal forms. Explain WHY
 | a table fails a normal form. Complete all sections.

Secure | Extension: Given an unnormalised flat-file table with
 | multiple problems, normalise to 3NF step by step. Write
 | the complete CREATE TABLE statements with primary and
 | foreign key declarations. Design a normalised database
 | for a hospital system with 4+ entities.

Resources Needed:

- Slide deck (foreign key diagrams, normalisation examples)
- Pre-built database in Thonny for JOIN practice
- Worksheet 3A (one per student)
- Normalisation flowchart handout
- Whiteboard for drawing table structures

LESSON 3.2 — DATABASE DESIGN TASK

Topic: Complete Database Design with 3+ Linked Tables, Practical SQL

Duration: 40 minutes

Lesson Type: Project-Based Practical Task

Syllabus Ref: CIE 0478 Topic 9 (consolidation)

Learning Objective:

By the end of this lesson, all students will be able to design a complete relational database with at least 3 linked tables for a given scenario, create an ERD, identify primary and foreign keys, normalise to at least 2NF, and write SQL queries to extract meaningful information.

Key Vocabulary:

- design = 设计 (shè jì)
- scenario = 场景/情景 (chǎng jǐng / qíng jǐng)
- implement = 实现/实施 (shí xiàn / shí shī)
- linked table = 链接表 (liàn jiē biǎo)
- query = 查询 (chá xún)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "A hospital needs a database to track patients, doctors,

- | and appointments. What tables would you create? What
- | information would each table store?"
- | Quick brainstorm in notebook. Share one idea with partner.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Database Design Process — STEP BY STEP:
 - | Step 1: Read the scenario carefully
 - | Step 2: Identify the ENTITIES (nouns in the description)
 - | Step 3: List ATTRIBUTES for each entity (fields)
 - | Step 4: Identify the PRIMARY KEY for each entity
 - | Step 5: Identify RELATIONSHIPS between entities
 - | Step 6: Draw the ERD
 - | Step 7: Check for normalisation (at least 2NF)
 - | Step 8: Write SQL queries to test the design
- | (Slide 2) Worked Example — Online Shop:
 - | Step 1: Scenario — "An online shop sells products to
 - | customers. Each customer can place many orders.
 - | Each order can contain many products."
 - | Step 2: Entities: Customer, Order, Product
 - | Step 3: Attributes:
 - | Customer: CustomerID, Name, Email, Address
 - | Order: OrderID, CustomerID(FK), OrderDate, TotalAmount
 - | Product: ProductID, ProductName, Price, Category
 - | Step 4: Primary Keys:
 - | Customer: CustomerID
 - | Order: OrderID
 - | Product: ProductID
 - | Step 5: Relationships:
 - | Customer -> Order: 1:M
 - | Order -> Product: M:N (needs link table)
 - | Step 6: ERD drawn on board
 - | Step 7: Normalisation check:
 - | All tables have single-field PKs -> automatically 2NF
 - | No transitive dependencies -> 3NF OK
 - | Step 8: SQL queries:
 - | -- Show all orders for customer C001
 - | SELECT * FROM Order WHERE CustomerID = 'C001'

- | -- Show products in order O100
- | SELECT Product.ProductName, OrderItem.Quantity
- | FROM OrderItem
- | INNER JOIN Product ON OrderItem.ProductID = Product.ProductID
- | WHERE OrderItem.OrderID = 'O100'

12-25 min | GUIDED PRACTICE

- | Whole-class: Hospital Database Design
- | Scenario: "A hospital needs to track patients, doctors,
- | and appointments. Each patient can have many appointments.
- | Each doctor can have many appointments. An appointment is
- | for one patient with one doctor."
- | Step 1: Entities: Patient, Doctor, Appointment
- | Step 2: Attributes:
 - | Patient: PatientID, Name, DateOfBirth, Phone
 - | Doctor: DoctorID, Name, Speciality, Phone
 - | Appointment: ApptID, PatientID(FK), DoctorID(FK),
 - | Date, Time, Notes
- | Step 3: Relationships:
 - | Patient -> Appointment: 1:M
 - | Doctor -> Appointment: 1:M
- | Step 4: Draw ERD on whiteboard together.
- | Step 5: Write SQL queries together:
 - | -- Find all appointments for patient P001
 - | SELECT * FROM Appointment WHERE PatientID = 'P001'
 - | -- Find all appointments with Dr. Wang
 - | SELECT * FROM Appointment
 - | INNER JOIN Doctor ON Appointment.DoctorID = Doctor.DoctorID
 - | WHERE Doctor.Name = 'Dr. Wang'

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 3B — Database Design Task
- | Students choose ONE scenario and complete full design:
 - | Scenario A: Library system (Books, Members, Loans)
 - | Scenario B: Online food delivery (Restaurants, Customers, Orders)
 - | Scenario C: School system (Students, Teachers, Classes)
- | For chosen scenario, students must:
 - | - List entities and attributes

- | - Identify primary keys
- | - Draw an ERD
- | - Identify relationships with cardinality
- | - Write 3 SQL queries for the database
- | (Worksheet 3B distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: Gallery Walk — students display their ERDs on desks.
- | Walk around to see 2 other students' designs. Use sticky notes to give one compliment and one suggestion.
- | Exit ticket: "What is the most important thing to remember when designing a relational database?"
- | Homework: Finish database design if not complete. Write 2 additional SQL queries for your chosen scenario, including one that uses INNER JOIN.

EAL Support:

- Step-by-step design checklist (bilingual) on desk
- Sentence frames for each design step
- Vocabulary card with design-related terms
- Colour-coded entity cards: cut out and physically arrange before drawing
- Model answer partially revealed for reference

Differentiation:

Emerging | Complete Scenario C (school system — most familiar context).

- | Provide pre-filled entity and attribute lists. Student
- | draws ERD and labels relationships only. Complete SQL
- | with sentence frames and keyword bank.

Developing | Choose any scenario. Complete all design steps independently.

- | Write SQL queries from scratch. Check normalisation.

Secure | Extension: Complete TWO scenarios. For each, write CREATE

- | TABLE statements with PRIMARY KEY and FOREIGN KEY
- | constraints. Include a M:N relationship with proper link
- | table. Write 5+ SQL queries including nested conditions.

Resources Needed:

- Slide deck (design process, worked example)
- Scenario description cards (one per student)
- Blank A4 paper for ERD drawing
- Rulers and coloured pencils

- Worksheet 3B
- Sticky notes for gallery walk
- SQL reference cards

WORKSHEET 3A — FOREIGN KEYS, JOINS & NORMALISATION

Name: _____ Class: _____ Date: _____

Learning Objective: I can explain foreign keys, write INNER JOIN queries, and identify normal forms in database tables.

Bilingual Support:

- foreign key = 外键 (wài jiàn)
- INNER JOIN = 内连接 (nèi lián jiē)
- normalisation = 规范化 (guī fàn huà)
- 1NF = 第一范式 (dì yī fàn shì)
- 2NF = 第二范式 (dì èr fàn shì)
- 3NF = 第三范式 (dì sān fàn shì)
- repeating group = 重复组 (chóng fù zǔ)
- dependency = 依赖 (yī lài)

SECTION A: Identify Foreign Keys [4 marks]

Study this database design:

Customers Table: Orders Table:

CustomerID	Name	OrderID	CustomerID	Amount
C001	Zhang	O100	C001	250
C002	Li	O101	C002	89
C003	Wang	O102	C001	156

56. What is the primary key in the Customers table? _____ [1]

57. What is the primary key in the Orders table? _____ [1]

58. What is the foreign key in the Orders table? _____ [1]

4. Explain what the foreign key does in this database.

_____ [1]

SECTION B: Write INNER JOIN Queries [8 marks]

Use these tables:

Students: Enrolments:

StudentID	Name	StudentID	CourseID	Grade
S001	Chen	S001	CS101	A
S002	Wang	S001	MA102	B

S003	Liu	S002	CS101	A
+-----+-----+		S003	PH103	C

59. Write a query to show student names alongside their course IDs.

_____ [2]

60. Write a query to show the names of all students enrolled in CS101.

_____ [2]

61. Write a query to show all student names and grades for course MA102.

_____ [2]

62. Write a query to show how many students are enrolled in each course.

(Hint: you only need the Enrolments table for this.)

_____ [2]

SECTION C: Normalisation [8 marks]

9. This table is NOT in 1NF. Explain why and fix it.

StudentSubjects:

StudentID	Name	Subject1	Subject2
S001	Zhang Wei	CS	Math
S002	Li Na	Physics	CS

Problem: _____ [1]

Fixed design: _____ [3]

63. This table is in 1NF but NOT in 2NF. Explain why and fix it.

Enrolment:

StudentID	CourseID	Grade	CourseName
-----------	----------	-------	------------

(Primary key = StudentID + CourseID)

Problem: _____ [1]

Fixed design: _____ [3]

WORKSHEET 3B — DATABASE DESIGN TASK

Name: _____ Class: _____ Date: _____

Learning Objective: I can design a complete relational database with 3+ linked tables for a real-world scenario, create an ERD, and write SQL queries.

Bilingual Support:

- design = 设计 (shè jì)
- entity = 实体 (shí tǐ)
- attribute = 属性 (shǔ xìng)
- primary key = 主键 (zhǔ jiàn)

- foreign key = 外键 (wài jiàn)
- relationship = 关系 (guān xì)
- ERD = 实体关系图 (shí tǐ guān xì tú)

INSTRUCTIONS: Choose ONE scenario below and complete ALL sections.

SCENARIO A: Library System

A school library needs to track books, members (borrowers), and loans.

Each book can be loaned many times. Each member can borrow many books.

A loan links one member to one book at a time.

SCENARIO B: Online Food Delivery

A food delivery platform needs to track restaurants, customers, and orders.

Each restaurant receives many orders. Each customer can place many orders.

Each order is from one customer to one restaurant.

SCENARIO C: School Timetable System

A school needs to track students, teachers, and classes.

Each teacher teaches many classes. Each class has one teacher.

Each student attends many classes. Each class has many students.

I choose Scenario: _____ (circle A, B, or C)

SECTION A: Identify Entities and Attributes [6 marks]

List at least 3 entities for your scenario. For each, list at least 3 attributes and identify the primary key.

Entity 1: _____

Attributes: _____ [1]

Primary Key: _____ [1]

Entity 2: _____

Attributes: _____ [1]

Primary Key: _____ [1]

Entity 3: _____

Attributes: _____ [1]

Primary Key: _____ [1]

SECTION B: Draw an ERD [6 marks]

Draw your ERD in the space below. Use rectangles for entities and label the relationships with 1:M, 1:1, or M:N. Label primary keys.

[DRAW ERD HERE]

SECTION C: Identify Relationships [3 marks]

Explain each relationship in your database:

Relationship 1: _____ [1]

Relationship 2: _____ [1]

Relationship 3 (if applicable): _____ [1]

SECTION D: Write SQL Queries [6 marks]

Write 3 useful SQL queries for your database.

Query 1 — Show all records from one table:

_____ [2]

Query 2 — Filter with WHERE:

_____ [2]

Query 3 — Use INNER JOIN to combine two tables:

_____ [2]

SECTION E: Normalisation Check [3 marks]

Is your database design in 3NF? Explain why or why not.

_____ [3]

ANSWER KEY — WEEK 3

WORKSHEET 3A:

SECTION A:

1. CustomerID [1]

2. OrderID [1]

3. CustomerID [1]

64. The foreign key (CustomerID in Orders table) links each order to the customer who placed it [1]. It references the primary key in the Customers table [1]. Accept equivalent explanation.

SECTION B:

5. SELECT Students.Name, Enrolments.CourseID

FROM Students

INNER JOIN Enrolments ON Students.StudentID = Enrolments.StudentID [2]

(1 mark for correct SELECT, 1 mark for correct INNER JOIN)

6. SELECT Students.Name

FROM Students

INNER JOIN Enrolments ON Students.StudentID = Enrolments.StudentID

WHERE Enrolments.CourseID = 'CS101' [2]

Accept: SELECT Name FROM Students WHERE StudentID IN

(SELECT StudentID FROM Enrolments WHERE CourseID = 'CS101') [2]

7. SELECT Students.Name, Enrolments.Grade

FROM Students

INNER JOIN Enrolments ON Students.StudentID = Enrolments.StudentID

WHERE Enrolments.CourseID = 'MA102' [2]

65. SELECT CourseID, COUNT(*) FROM Enrolments GROUP BY CourseID [2]

Accept: SELECT * FROM Enrolments WHERE CourseID = 'CS101'

combined with visual counting explanation [1]

SECTION C:

66. Problem: There is a repeating group (Subject1, Subject2) [1].

A repeating group means the table is NOT in 1NF.

Fixed design: [3]

Two tables needed:

Student(StudentID, Name) with PK = StudentID [1]

StudentSubject(StudentID, Subject) with PK = StudentID + Subject [1]

Or: StudentSubject as a separate table with individual rows:

S001, CS and S001, Math as separate records [1].

67. Problem: CourseName depends only on CourseID (part of the composite key), not the whole key (StudentID + CourseID) [1]. This is a partial dependency, so the table is NOT in 2NF.

Fixed design: [3]

Enrolment(StudentID, CourseID, Grade) with PK = StudentID + CourseID [1]

Course(CourseID, CourseName) with PK = CourseID [1]

Explanation: CourseName moved to Course table where CourseID is the full primary key [1].

WORKSHEET 3B:

SECTION A: (Accept any valid design. Example answers below.)

Scenario A (Library):

Entity 1: Book — BookID, Title, Author, ISBN; PK = BookID [2]

Entity 2: Member — MemberID, Name, Email, JoinDate; PK = MemberID [2]

Entity 3: Loan — LoanID, BookID(FK), MemberID(FK), LoanDate, ReturnDate;

PK = LoanID [2]

Scenario B (Food Delivery):

Entity 1: Restaurant — RestID, Name, Cuisine, Phone; PK = RestID [2]

Entity 2: Customer — CustID, Name, Address, Phone; PK = CustID [2]

Entity 3: Order — OrderID, CustID(FK), RestID(FK), OrderDate, Total;
PK = OrderID [2]

Scenario C (School):

Entity 1: Student — StudentID, Name, YearGroup; PK = StudentID [2]

Entity 2: Teacher — TeacherID, Name, Subject; PK = TeacherID [2]

Entity 3: Class — ClassID, TeacherID(FK), Room, Period; PK = ClassID [2]

Entity 4: Enrolment — StudentID(FK), ClassID(FK); PK = both fields [2]

(3 entities minimum, 4 acceptable)

SECTION B: [6 marks]

ERD marking: 2 marks for correct entities shown, 2 marks for correct relationship types and cardinality labels, 2 marks for correct PK labels.

Deduct 1 if M:N relationship missing link table.

SECTION C: [3 marks]

Relationship explanations should include correct cardinality.

1 mark per relationship explanation.

SECTION D: [6 marks]

2 marks per query. 1 mark for correct structure, 1 mark for logic.

Accept any valid queries for the student's chosen design.

SECTION E: [3 marks]

Award marks for:

- Stating whether design is in 3NF [1]
- Explaining that all tables have a primary key and no repeating groups (1NF) [1]
- Explaining no partial or transitive dependencies (2NF/3NF) [1]

WEEK 4: BOOLEAN LOGIC — BASIC GATES

Overview: This week introduces Boolean logic — the mathematical foundation of all digital circuits. Students learn the three basic logic gates (AND, OR, NOT), build and interpret truth tables with 2 inputs, translate between logic expressions and circuit diagrams, and apply Boolean logic to real-world problems like security systems. Cambridge IGCSE uses standard gate symbols and expects students to complete truth tables and draw circuits. This is entirely new content not covered in Year 9.

Syllabus Ref: CIE 0478 Topic 10.1, 10.2

LESSON 4.1 — INTRODUCTION TO BOOLEAN LOGIC: AND, OR, NOT

Topic: AND Gate, OR Gate, NOT Gate, Truth Tables with 2 Inputs, Logic Expressions

Duration: 40 minutes

Lesson Type: Conceptual Introduction with Practical Table Completion

Syllabus Ref: CIE 0478 Topic 10.1

Learning Objective:

By the end of this lesson, all students will be able to describe the function of AND, OR, and NOT gates, complete truth tables for these gates with 2 inputs, write simple logic expressions, and draw circuit diagrams from expressions.

Key Vocabulary:

- Boolean logic = 布尔逻辑 (bù ěr luó jí)
- logic gate = 逻辑门 (luó jí mén)
- truth table = 真值表 (zhēn zhí biǎo)
- AND gate = 与门 (yǔ mén)
- OR gate = 或门 (huò mén)
- NOT gate = 非门 (fēi mén)
- input = 输入 (shū rù)
- output = 输出 (shū chū)
- logic expression = 逻辑表达式 (luó jí biǎo dá shì)
- circuit diagram = 电路图 (diàn lù tú)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You have two light switches controlling one light.
- | Switch A AND Switch B must both be ON for the light to work.
- | How many possible combinations of on/off are there?
- | Write them all down."
- | (Answer: 4 combinations — A=0,B=0; A=0,B=1; A=1,B=0; A=1,B=1)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) What is Boolean logic?
 - | - Named after George Boole (1815–1864), an English mathematician
 - | - A system of algebra using only TWO values: TRUE/FALSE or 1/0
 - | - The foundation of ALL computer circuits — every CPU, every smartphone, every program uses Boolean logic
 - | - In computers: 1 = ON (voltage present), 0 = OFF (no voltage)
- | (Slide 2) AND Gate:
 - | Symbol: A ---[AND]--- Output

| B ---

| Rule: Output is 1 ONLY if BOTH inputs are 1

| Real-world: "You can enter the building IF you have a

| card AND the door is unlocked"

| Truth Table:

A	B	A AND B
0	1	0
1	0	0

| (Slide 3) OR Gate:

| Symbol: A ---[OR]--- Output

| B ---

| Rule: Output is 1 if AT LEAST ONE input is 1

| Real-world: "The alarm sounds IF motion detected OR

| window broken"

| Truth Table:

A	B	A OR B
0	1	1
1	0	1

| (Slide 4) NOT Gate:

| Symbol: A ---[NOT]--- Output

| Rule: Output is the OPPOSITE of the input

| Real-world: "The light is OFF when the switch is ON"

| Truth Table:

A	NOT A
0	1
1	0

| (Slide 5) Logic Expressions:

| AND is written as: $A \cdot B$ or "A AND B"

| OR is written as: $A + B$ or "A OR B"

| NOT is written as: $\bar{A}$ or "NOT A"

| Example expression: $Q = A \cdot B$

| This means: Output Q equals A AND B

| Example expression: $Q = A + \bar{B}$

| This means: Output Q equals A OR (NOT B)

12-25 min | GUIDED PRACTICE

| Exercise 1 (whole class):

| Complete the truth table for $Q = A \text{ AND } \bar{B}$

| | A | B | $\bar{B}$ | A AND $\bar{B}$ |

| | 0 | 0 | 1 | 0 |

| | 0 | 1 | 0 | 0 |

| | 1 | 0 | 1 | 1 |

| | 1 | 1 | 0 | 0 |

| Exercise 2 (pairs):

| Complete the truth table for $Q = (A + B) \text{ AND } \bar{C}$

| | A | B | C | A+B | $\bar{C}$ | (A+B) AND $\bar{C}$ |

| | 0 | 0 | 0 | 0 | 1 | 0 |

| | 0 | 0 | 1 | 0 | 0 | 0 |

| | 0 | 1 | 0 | 1 | 1 | 1 |

| | 0 | 1 | 1 | 1 | 0 | 0 |

| | 1 | 0 | 0 | 1 | 1 | 1 |

| | 1 | 0 | 1 | 1 | 0 | 0 |

| | 1 | 1 | 0 | 1 | 1 | 1 |

| | 1 | 1 | 1 | 1 | 0 | 0 |

| Exercise 3 (individual):

| "Draw a circuit diagram for $Q = A \text{ AND } (\text{NOT } B)$ "

| -> A goes to AND gate directly

| -> B goes through NOT gate first, then to AND gate

| -> AND gate output is Q

25-35 min | INDEPENDENT PRACTICE

| Worksheet 4A — Boolean Logic: AND, OR, NOT Gates

| Section A: Complete truth tables for AND, OR, NOT

| Section B: Complete truth tables for compound expressions

| Section C: Draw circuit diagrams for given expressions

| (Worksheet 4A distributed — see below)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Gate Speed Round"

| Teacher calls out gate type and inputs, students call output.

| "AND, 1 and 0" -> "0!"

| "OR, 0 and 1" -> "1!"

| "NOT, 0" -> "1!"

| "AND, 1 and 1" -> "1!"

| "OR, 0 and 0" -> "0!"

- | Thumbs up/middle/down for confidence level.
- | Homework: Complete the truth table for $Q = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$.
- | Show all working columns (A AND B, NOT C, then final output).

EAL Support:

- Boolean logic reference card with gate symbols, names, and truth tables
- Colour-code: AND = blue, OR = green, NOT = red
- Sentence frames: "When A is ____ and B is ____, the output is ____ because..."
- Visual gate symbol posters on wall
- Chinese cognate: "逻辑" (luó jí) = logic
- Physical card activity: use cards with 0 and 1 to simulate gates

Differentiation:

Emerging | Complete truth tables for single gates only (AND, OR, NOT separately). Use visual gate cards to physically trace inputs and outputs. Fill-in-the-blank for circuit diagrams.
| Complete Section A only.

Developing | Complete all truth tables including compound expressions.
| Draw circuit diagrams from expressions. Explain in sentences why a particular output is 0 or 1.

Secure | Extension: Design a circuit with 3 gates for a given real-world scenario. Write both the logic expression and draw the complete circuit. Create a truth table with all 8 rows for a 3-input circuit.

Resources Needed:

- Slide deck (gate symbols, truth tables, expressions)
- Boolean logic reference cards (laminated, one per student)
- Gate symbol posters for wall
- Worksheet 4A (one per student)
- Physical 0/1 cards for kinaesthetic learners
- Whiteboard for drawing circuits

LESSON 4.2 — COMBINING GATES & REAL-WORLD APPLICATIONS

Topic: Multiple Gates in Sequence, Circuit Drawing, Real-World Logic Applications

Duration: 40 minutes

Lesson Type: Practical with Design Tasks

Syllabus Ref: CIE 0478 Topic 10.2

Learning Objective:

By the end of this lesson, all students will be able to combine multiple logic gates to create circuits, draw accurate circuit diagrams, trace through multi-gate circuits to find outputs, and design simple logic circuits for real-world scenarios.

Key Vocabulary:

- combine = 组合 (zǔ hé)
- circuit = 电路 (diàn lù)
- trace = 追踪 (zhuī zōng)
- design = 设计 (shè jì)
- security system = 安全系统 (ān quán xì tǒng)
- alarm = 警报 (jǐng bào)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "A burglar alarm has two sensors: a door sensor (D) and a window sensor (W). The alarm sounds if EITHER sensor detects an intruder. Draw the logic circuit and write the expression."
- | Quick sketch in notebook. Share with partner.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Combining Gates:
 - | Outputs from one gate can be inputs to another.
 - | Example: $Q = (A \text{ AND } B) \text{ OR } C$
 - | A ---[AND]--- ---[OR]--- Q
 - | B --- C ---
 - | Step 1: $A \text{ AND } B \rightarrow$ intermediate result
 - | Step 2: $(A \text{ AND } B) \text{ OR } C \rightarrow$ final output Q
- | (Slide 2) Tracing through a circuit:
 - | Given inputs $A=1, B=0, C=1$:
 - | Step 1: $A \text{ AND } B = 1 \text{ AND } 0 = 0$
 - | Step 2: $0 \text{ OR } C = 0 \text{ OR } 1 = 1$
 - | Therefore $Q = 1$
- | (Slide 3) Real-world: Security System
 - | Scenario: "A safe has a fingerprint scanner (F) and a PIN keypad (P). The safe opens ONLY if BOTH are correct."
 - | Expression: $Q = F \text{ AND } P$
 - | (This is just an AND gate!)

- | (Slide 4) Real-world: Car Alarm
- | Scenario: "A car alarm sounds if:
- | (door forced AND engine off) OR (window broken)"
- | Let D = door forced, E = engine on, W = window broken
- | Expression: $Q = (D \text{ AND NOT } E) \text{ OR } W$
- | D ---[AND]--- [OR]--- Q (alarm)
- | E [NOT] W ---
- | (Slide 5) Drawing conventions:
- | - Inputs on the LEFT
- | - Outputs on the RIGHT
- | - Gates in the middle
- | - Straight lines, right-angle corners
- | - Label all inputs and outputs
- | - Use Cambridge standard symbols (ANSI rectangular symbols)

12-25 min | **GUIDED PRACTICE**

- | Exercise 1 (whole class):
- | "A sprinkler system turns on if:
- | (temperature is high) AND (smoke is detected)"
- | Let T = temperature high, S = smoke detected
- | Expression: $Q = T \text{ AND } S$
- | Draw circuit together on whiteboard.
- | Exercise 2 (pairs):
- | "A lift door opens if:
- | (at correct floor AND door button pressed) OR emergency stop"
- | Let F = at floor, B = button pressed, E = emergency
- | Expression: $Q = (F \text{ AND } B) \text{ OR } E$
- | Trace through with F=1, B=1, E=0 -> Q=1
- | Trace through with F=1, B=0, E=0 -> Q=0
- | Exercise 3 (individual):
- | "A classroom projector turns on if:
- | (teacher present OR remote pressed) AND (power available)"
- | Let T = teacher present, R = remote pressed, P = power
- | Expression: $Q = (T \text{ OR } R) \text{ AND } P$
- | Draw circuit and trace with T=0, R=1, P=1 -> Q=1

25-35 min | **INDEPENDENT PRACTICE**

- | Worksheet 4B — Combining Gates & Circuit Design

- | Section A: Trace through given multi-gate circuits
- | Section B: Draw circuits for given expressions
- | Section C: Design circuits for 3 real-world scenarios
- | (Worksheet 4B distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Circuit Design Challenge"
- | "Design a circuit for a smart classroom light:
- | Turns ON if: (motion detected AND light level low) OR
- | manual override switch is ON"
- | Students sketch on mini-whiteboard, hold up.
- | Teacher reviews best examples.
- | Homework: A vending machine dispenses a drink if:
- | (correct money inserted AND drink selected) AND
- | (drink is in stock)
- | Draw the circuit diagram and write the logic expression.
- | Create a truth table for all 8 possible input combinations.

EAL Support:

- Circuit drawing template with pre-drawn gate outlines
- Sentence frames for tracing: "First, ___ AND ___ = ___. Then, ___ OR ___ = ___. So the output is ___."
- Real-world scenario vocabulary card
- Colour-code: motion sensors (blue), alarms (red), power (green)
- Visual step-by-step tracing guide

Differentiation:

Emerging | Trace through pre-drawn circuits with input values given.

- | Use template with gate outlines to draw circuits.
- | Focus on 2-gate circuits only. Complete Sections A and B.

Developing | Draw complete circuits from expressions. Trace through

- | with different input values. Design circuits for
- | real-world scenarios with up to 3 gates.

Secure | Extension: Design a circuit for a traffic light system

- | with 4 inputs (car present, pedestrian waiting, timer
- | expired, emergency vehicle) using AND, OR, NOT gates.
- | Write the complete expression and create full truth table.

Resources Needed:

- Slide deck (combined circuits, real-world examples)

- Circuit drawing templates
- Worksheet 4B (one per student)
- Mini-whiteboards for plenary
- Gate symbol reference posters
- Rulers for drawing neat circuits

WORKSHEET 4A — BOOLEAN LOGIC: AND, OR, NOT GATES

Name: _____ Class: _____ Date: _____

Learning Objective: I can complete truth tables for AND, OR, and NOT gates and understand how each gate works.

Bilingual Support:

- Boolean logic = 布尔逻辑 (bù ěr luó jí)
- truth table = 真值表 (zhēn zhí biǎo)
- AND gate = 与门 (yǔ mén)
- OR gate = 或门 (huò mén)
- NOT gate = 非门 (fēi mén)
- input = 输入 (shū rù)
- output = 输出 (shū chū)
- logic expression = 逻辑表达式 (luó jí biǎo dá shì)

SECTION A: Complete the Basic Truth Tables [6 marks]

1. AND Gate: $Q = A \text{ AND } B$

A	B	$Q = A \text{ AND } B$
0	0	[1]
0	1	[1]
1	0	[1]
1	1	[1]

2. OR Gate: $Q = A \text{ OR } B$

A	B	$Q = A \text{ OR } B$
0	0	[1]
0	1	[1]
1	0	[1]
1	1	[1]

3. NOT Gate: $Q = \text{NOT } A$

A	Q = NOT A
---	-----------

0	[1]
---	-----

1	[1]
---	-----

SECTION B: Complete Compound Truth Tables [8 marks]

4. $Q = A \text{ AND } (\text{NOT } B)$

Complete all columns:

A	B	NOT B	A AND (NOT B)
---	---	-------	---------------

0	0	[1]	[1]
---	---	-----	-----

0	1	[1]	[1]
---	---	-----	-----

1	0	[1]	[1]
---	---	-----	-----

1	1	[1]	[1]
---	---	-----	-----

5. $Q = (A \text{ OR } B) \text{ AND } C$ (3 inputs = 8 rows)

A	B	C	A OR B	(A OR B) AND C
---	---	---	--------	----------------

0	0	0	[1]	[1]
---	---	---	-----	-----

0	0	1	[1]	[1]
---	---	---	-----	-----

0	1	0	[1]	[1]
---	---	---	-----	-----

0	1	1	[1]	[1]
---	---	---	-----	-----

1	0	0	[1]	[1]
---	---	---	-----	-----

1	0	1	[1]	[1]
---	---	---	-----	-----

1	1	0	[1]	[1]
---	---	---	-----	-----

1	1	1	[1]	[1]
---	---	---	-----	-----

SECTION C: Draw Circuit Diagrams [6 marks]

6. Draw the circuit for $Q = A \text{ AND } B$

[Draw your circuit here using standard gate symbols]

[2 marks — 1 for correct gates, 1 for correct connections]

7. Draw the circuit for $Q = A \text{ OR } (\text{NOT } B)$

[Draw your circuit here]

[2 marks — 1 for correct gates including NOT, 1 for connections]

8. Draw the circuit for $Q = (A \text{ AND } B) \text{ OR } (A \text{ AND } C)$

[Draw your circuit here]

[2 marks — 1 for correct gates, 1 for correct connections]

WORKSHEET 4B — COMBINING GATES & CIRCUIT DESIGN

Name: _____ Class: _____ Date: _____

Learning Objective: I can combine multiple logic gates, trace through circuits, and design circuits for real-world scenarios.

Bilingual Support:

- combine = 组合 (zǔ hé)
- circuit = 电路 (diàn lù)
- trace = 追踪 (zhuī zōng)
- design = 设计 (shè jì)
- security system = 安全系统 (ān quán xì tǒng)
- sensor = 传感器 (chuán gǎn qì)

SECTION A: Trace Through These Circuits [8 marks]

1. Given this circuit and inputs $A=1$, $B=0$, $C=1$:

A ---[AND]--- ---[OR]--- Q

B --- C ---

Step 1: $A \text{ AND } B = \underline{\quad} \text{ AND } \underline{\quad} = \underline{\quad}$ [1]

Step 2: (result) OR C = $\underline{\quad}$ OR $\underline{\quad} = \underline{\quad}$ [1]

Final output Q = $\underline{\quad}$ [1]

2. Given this circuit and inputs $A=0$, $B=1$:

A ---[NOT]--- ---[AND]--- Q

Step 1: NOT A = NOT $\underline{\quad} = \underline{\quad}$ [1]

Step 2: (NOT A) AND B = $\underline{\quad}$ AND $\underline{\quad} = \underline{\quad}$ [1]

Final output Q = $\underline{\quad}$ [1]

68. Complete this truth table for the circuit $Q = (A \text{ OR } B) \text{ AND } (\text{NOT } C)$:

A	B	C	A OR B	NOT C	Q
0	0	0	[1]	[1]	[1]
0	0	1	[1]	[1]	[1]
0	1	0	[1]	[1]	[1]
0	1	1	[1]	[1]	[1]

SECTION B: Draw Circuits from Expressions [6 marks]

4. Draw the circuit for $Q = A \text{ AND } (B \text{ OR } C)$

[Draw here]

5. Draw the circuit for $Q = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$

[Draw here]

6. Draw the circuit for $Q = \text{NOT } (A \text{ AND } B)$

(This is a NAND gate — we will learn this next lesson!)

[Draw here]

SECTION C: Design Circuits for Real-World Scenarios [6 marks]

For each scenario:

- (i) List the inputs and what 0/1 means for each
- (ii) Write the logic expression
- (iii) Draw the circuit diagram

69. A smoke detector alarm sounds if: (smoke detected) OR (manual test pressed)

Inputs: _____ [1]

Expression: _____ [1]

[Draw circuit here]

70. A smart door lock opens if: (correct fingerprint) AND (door handle turned)

Inputs: _____ [1]

Expression: _____ [1]

[Draw circuit here]

ANSWER KEY — WEEK 4

WORKSHEET 4A:

SECTION A:

1. AND Gate: 0, 0, 0, 1 [1 mark each = 4 marks]

2. OR Gate: 0, 1, 1, 1 [1 mark each = 4 marks]

3. NOT Gate: 1, 0 [1 mark each = 2 marks]

SECTION B:

4. $Q = A \text{ AND } (\text{NOT } B)$:

A	B	NOT B	A AND (NOT B)	
0	0	1	0	[1]
0	1	0	0	[1]
1	0	1	1	[1]
1	1	0	0	[1]

5. $Q = (A \text{ OR } B) \text{ AND } C$:

A	B	C	A OR B	(A OR B) AND C	
0	0	0	0	0	[1]
0	0	1	0	0	[1]
0	1	0	1	0	[1]
0	1	1	1	1	[1]
1	0	0	1	0	[1]
1	0	1	1	1	[1]
1	1	0	1	0	[1]
1	1	1	1	1	[1]

SECTION C:

6. A and B both connect to AND gate. AND output is Q. [2]

71. A connects to AND. B goes through NOT gate first, then to AND.

AND output is Q. [2]

72. A and B go to first AND gate. A and C go to second AND gate.

Both AND outputs go to OR gate. OR output is Q. [2]

Accept equivalent correct layouts.

WORKSHEET 4B:

SECTION A:

1. Step 1: $1 \text{ AND } 0 = 0$ [1]

Step 2: $0 \text{ OR } 1 = 1$ [1]

Final output $Q = 1$ [1]

2. Step 1: $\text{NOT } 0 = 1$ [1]

Step 2: $1 \text{ AND } 1 = 1$ [1]

Final output $Q = 1$ [1]

3. $Q = (A \text{ OR } B) \text{ AND } (\text{NOT } C)$:

A	B	C	A OR B	NOT C	Q
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0

SECTION B:

73. A to one input of AND. B and C to OR gate. OR output to other AND input.

AND output is Q. [2]

74. A and B to AND gate. C to NOT gate. AND output and NOT output to OR gate.

OR output is Q. [2]

75. A and B to AND gate. AND output to NOT gate. NOT output is Q. [2]

SECTION C:

7. Inputs: S = smoke detected (1 = detected, 0 = no smoke),

T = test pressed (1 = pressed, 0 = not pressed) [1]

Expression: $Q = S \text{ OR } T$ [1]

Circuit: S and T to OR gate. OR output is Q (alarm). [1]

Accept equivalent descriptions and diagrams.

76. Inputs: F = fingerprint correct (1 = correct, 0 = incorrect),

H = handle turned (1 = turned, 0 = not turned) [1]

Expression: $Q = F \text{ AND } H$ [1]

Circuit: F and H to AND gate. AND output is Q (lock opens). [1]

Accept equivalent descriptions and diagrams.

WEEK 5: BOOLEAN LOGIC — ADVANCED GATES

Overview: This week extends Boolean logic to the advanced gates: NAND, NOR, and XOR. Students learn that these gates are combinations of the basic gates, build truth tables with 3 inputs, and discover that NAND is a "universal gate" — capable of creating any other gate. The week culminates with half-adders, the building blocks of computer arithmetic, showing students how logic gates directly enable everything CPUs do. This is challenging, abstract content — pacing is deliberate with multiple worked examples.

Syllabus Ref: CIE 0478 Topic 10.3, 10.4, 10.5, 10.6

Prerequisites:

- Week 4 content (AND, OR, NOT gates, truth tables, expressions, circuits)

LESSON 5.1 — NAND, NOR, XOR GATES & 3-INPUT TRUTH TABLES

Topic: NAND Gate, NOR Gate, XOR Gate, Truth Tables with 3 Inputs, Half-Adders

Duration: 40 minutes

Lesson Type: Conceptual with Practical Table Completion

Syllabus Ref: CIE 0478 Topic 10.3, 10.4

Learning Objective:

By the end of this lesson, all students will be able to describe NAND, NOR, and XOR gates, complete their truth tables, build truth tables with 3 inputs, and explain how a half-adder works using AND, OR, and XOR gates.

Key Vocabulary:

- NAND gate = 与非门 (yǔ fēi mén) — "AND-NOT gate"
- NOR gate = 或非门 (huò fēi mén) — "OR-NOT gate"
- XOR gate = 异或门 (yì huò mén) — "exclusive OR gate"
- exclusive = 排他的/独家的 (pái tā de / dú jiā de)
- universal gate = 通用门 (tōng yòng mén)
- half-adder = 半加器 (bàn jiā qì)
- sum = 和 (hé)
- carry = 进位 (jìn wèi)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "An AND gate outputs 1 only when both inputs are 1.
- | What if we put a NOT gate AFTER an AND gate? What would
- | the output be for each combination of inputs?"
- | Write predictions in notebook. Think-Pair-Share.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) NAND Gate (NOT-AND):
- | Symbol: Same as AND but with a small circle at the output
- | Rule: Output is 0 ONLY when BOTH inputs are 1
- | (Opposite of AND!)
- | Truth Table:
- | | A | B | A NAND B |
- | | 0 | 0 | 1 |
- | | 0 | 1 | 1 |
- | | 1 | 0 | 1 |
- | | 1 | 1 | 0 |
- | Real-world: "A safety system stays ON unless BOTH sensors

| confirm it is safe to proceed"

| (Slide 2) NOR Gate (NOT-OR):

| Symbol: Same as OR but with a small circle at the output

| Rule: Output is 1 ONLY when BOTH inputs are 0

| (Opposite of OR!)

| Truth Table:

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

| (Slide 3) XOR Gate (Exclusive OR):

| Symbol: Like OR but with extra curved line at input

| Rule: Output is 1 when inputs are DIFFERENT

| "Exclusive" means excluding the case where both are 1

| Truth Table:

A	B	A XOR B
0	1	1
1	0	1
1	1	0

| Real-world: "A light controlled by two switches — you can

| turn it on or off from EITHER switch, but not

| if both are in the same position"

| (Slide 4) XOR in your home:

| The stairway light in many homes uses XOR logic!

| Switch at bottom + Switch at top.

| If both are up OR both are down, light is OFF.

| If they are different, light is ON.

| (Slide 5) Truth Tables with 3 Inputs:

| With 3 inputs (A, B, C), there are $2^3 = 8$ rows.

| Systematic order: 000, 001, 010, 011, 100, 101, 110, 111

12-25 min | GUIDED PRACTICE

| Exercise 1 (whole class):

| Complete truth table for $Q = (A \text{ NAND } B) \text{ OR } C$

A	B	C	A NAND B	Q
0	0	0	1	1

0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	0	1

| Exercise 2 (pairs):

| "A half-adder adds two single binary digits (bits).

| It produces two outputs: SUM and CARRY."

| SUM = A XOR B (1 when inputs are different)

| CARRY = A AND B (1 only when both are 1)

| Truth Table:

A	B	SUM	CARRY
0	1	1	0
1	0	1	0
1	1	0	1

| Example: 1 + 1 = 10 in binary!

| SUM = 0, CARRY = 1 -> result is "10" (which is 2 in decimal)

| Trace through with A=1, B=1:

| SUM = 1 XOR 1 = 0

| CARRY = 1 AND 1 = 1

| Result: 10 (binary for 2)

25-35 min | INDEPENDENT PRACTICE

| Worksheet 5A — NAND, NOR, XOR & 3-Input Truth Tables

| Section A: Complete truth tables for NAND, NOR, XOR

| Section B: Trace through 3-input circuits

| Section C: Half-adder trace and explanation

| (Worksheet 5A distributed — see below)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Advanced Gate Speed Round"

| "XOR, 1 and 0" -> "1!"

| "NAND, 1 and 1" -> "0!"

| "NOR, 0 and 0" -> "1!"

| "NAND, 0 and 1" -> "1!"

| "XOR, 1 and 1" -> "0!"

- | "Why is XOR useful?" -> "It tells us when things are different!"
- | Homework: Complete a truth table for $Q = (A \text{ NOR } B) \text{ AND } C$
- | with all 8 rows. Show all working.

EAL Support:

- Gate reference card with ALL 6 gates (AND, OR, NOT, NAND, NOR, XOR) and their truth tables (bilingual)
- Memory aids:
 NAND = "Not And" -> 与(yu)非(wei) = 与非门
 NOR = "Not Or" -> 或(huo)非(wei) = 或非门
 XOR = "Exclusive Or" -> 异(yi)或(huo) = 异或门
- Highlight the circle on NAND/NOR outputs — "the circle means NOT"
- Sentence frame: "NAND is the OPPOSITE of AND, so when AND gives ____, NAND gives ____."
- Physical gate cards for kinaesthetic practice

Differentiation:

- Emerging | Complete truth tables for NAND, NOR, XOR separately (not combined). Use gate reference card for all answers. Focus on recognising gate symbols only. Complete Section A only.
- Developing | Complete all truth tables including 3-input circuits. Trace through half-adder with guidance. Explain in own words what each gate does.
- Secure | Extension: Write the logic expression for a full adder (3 inputs: A, B, and carry-in). Create the complete truth table with 8 rows and 2 outputs. Research: Why are half-adders important in CPU design?

Resources Needed:

- Slide deck (NAND, NOR, XOR symbols and truth tables, half-adder)
- Complete gate reference cards (all 6 gates)
- Worksheet 5A (one per student)
- Half-adder diagram poster
- Physical gate symbol cards

LESSON 5.2 — NAND AS UNIVERSAL GATE & LOGIC CIRCUIT PROBLEMS

Topic: NAND as Universal Gate, Converting Between Expressions/Circuits/Truth Tables

Duration: 40 minutes

Lesson Type: Problem-Solving with Practical Design

Syllabus Ref: CIE 0478 Topic 10.5, 10.6

Learning Objective:

By the end of this lesson, all students will be able to explain why NAND is called a universal gate, build NOT, AND, and OR using only NAND gates, convert between logic expressions, circuit diagrams, and truth tables, and design circuits for complex real-world problems.

Key Vocabulary:

- universal gate = 通用门 (tong yong men)
- equivalent = 等效/等价的 (deng xiao / deng jia de)
- convert = 转换 (zhuan huan)
- equivalent circuit = 等效电路 (deng xiao dian lu)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You are on a desert island with unlimited NAND gates but
- | NO other gates. How could you make a NOT gate using only
- | NAND gates?"
- | Draw in notebook. (Hint: what if both inputs are the same?)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) NAND as Universal Gate:
- | - A "universal gate" can create ANY other logic gate
- | - NAND is the most commonly used universal gate
- | - This is why NAND gates are the building blocks of
- | modern CPUs and memory chips
- | (Slide 2) NOT from NAND:
- | Connect both inputs of a NAND gate together!
- | $A \text{ ---[NAND]--- } Q = \text{NOT } A$
- | Proof: If $A=0$: $0 \text{ NAND } 0 = 1$ OK
- | If $A=1$: $1 \text{ NAND } 1 = 0$ OK
- | (Slide 3) AND from NAND:
- | NAND followed by NOT (made from another NAND)
- | $A \text{ ---[NAND]---[NAND]--- } Q = A \text{ AND } B$
- | $B \text{ --- } \text{ ---}$
- | First NAND gives $\text{NOT}(A \text{ AND } B)$
- | Second NAND (with inputs joined) gives $\text{NOT}(\text{NOT}(A \text{ AND } B))$
- | $= A \text{ AND } B$
- | (Slide 4) OR from NAND (De Morgan's Law — advanced):

- | A ---[NAND]--- ---
- | --- ---[NAND]--- Q = A OR B
- | B ---[NAND]--- ---
- | This converts NOT A and NOT B into A OR B using NAND!
- | (Slide 5) Converting Between Representations:
- | Expression -> Circuit: Replace each operator with gate
- | Circuit -> Expression: Trace from inputs to output
- | Expression -> Truth Table: Evaluate for all input combos
- | Truth Table -> Expression: Find rows where output is 1
- | Example: Given truth table row where Q=1 at A=1,B=0:
- | Expression includes: A AND (NOT B)

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class):
- | "Build an OR gate using only NAND gates."
- | Step 1: NOT A using NAND (inputs joined)
- | Step 2: NOT B using NAND (inputs joined)
- | Step 3: NAND(NOT A, NOT B) = A OR B (by De Morgan's)
- | Exercise 2 (pairs):
- | Convert the expression $Q = A \text{ AND } (B \text{ OR } \bar{C})$ to:
- | (a) A circuit diagram
- | (b) A truth table (8 rows)
- | Exercise 3 (individual):
- | Given this truth table, find the expression:
- | | A | B | Q |
- | | 0 | 1 | 0 |
- | | 1 | 0 | 1 |
- | Answer: $Q = A$ (output matches A exactly!)

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 5B — NAND Universal Gate & Circuit Problems
- | Section A: Build NOT, AND, OR from NAND gates
- | Section B: Convert between expressions, circuits, truth tables
- | Section C: Design circuits for complex real-world problems
- | (Worksheet 5B distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Universal Gate Challenge"
- | "Using only NAND gates, how would you make:

- | (a) NOT (b) AND (c) OR"
- | Students sketch on mini-whiteboards.
- | Award points for correct designs.
- | Key message: "NAND gates are in EVERY computer chip you own.
- | Your phone's CPU contains billions of NAND gates!"
- | Homework: A security system has 3 inputs:
 - | M = motion detected, D = door open, W = window open
 - | The alarm sounds if: (M AND D) OR (M AND W) OR (D AND W)
 - | (This is: at least 2 sensors triggered)
 - | (a) Draw the circuit using AND, OR gates
 - | (b) Write the complete truth table (8 rows)
 - | (c) Simplify using your understanding

EAL Support:

- Universal gate concept: "万能门" — one gate can do everything
- Step-by-step conversion guide (bilingual flowchart)
- Sentence frame for conversions:

"To convert an expression to a circuit, I replace each ____ with the matching ____ and connect them in the same order."
- NAND building block cards — physically arrange to build circuits
- Visual summary: "NAND + NAND + NAND = any circuit you need"

Differentiation:

Emerging | Focus on recognising that NAND is "AND then NOT." Build

- | NOT from NAND only (simplest conversion). Use pre-drawn
- | circuits to label gate types. Complete Section A only.

Developing | Build NOT, AND, and OR from NAND. Convert between all

- | three representations (expression, circuit, truth table).
- | Complete all sections with independence.

Secure | Extension: Research and explain WHY NAND is preferred

- | over NOR as a universal gate in manufacturing. Design
- | a circuit for a 2-bit comparator (compares two 2-bit
- | binary numbers and outputs which is larger) using only
- | basic gates. Write all expressions and truth tables.

Resources Needed:

- Slide deck (NAND as universal gate, conversion methods)
- NAND gate building block templates
- Worksheet 5B (one per student)

- Mini-whiteboards for plenary challenge
- Conversion flowchart handout

WORKSHEET 5A — NAND, NOR, XOR GATES & 3-INPUT TRUTH TABLES

Name: _____ Class: _____ Date: _____

Learning Objective: I can complete truth tables for NAND, NOR, and XOR gates, work with 3-input truth tables, and understand half-adders.

Bilingual Support:

- NAND gate = 与非门 (yu fei men)
- NOR gate = 或非门 (huo fei men)
- XOR gate = 异或门 (yi huo men)
- universal gate = 通用门 (tong yong men)
- half-adder = 半加器 (ban jia qi)
- sum = 和 (he)
- carry = 进位 (jin wei)

SECTION A: Complete Truth Tables [6 marks]

1. NAND Gate: $Q = A \text{ NAND } B$

A	B	$Q = A \text{ NAND } B$
0	0	[1]
0	1	[1]
1	0	[1]
1	1	[1]

2. NOR Gate: $Q = A \text{ NOR } B$

A	B	$Q = A \text{ NOR } B$
0	0	[1]
0	1	[1]
1	0	[1]
1	1	[1]

3. XOR Gate: $Q = A \text{ XOR } B$

A	B	$Q = A \text{ XOR } B$
0	0	[1]
0	1	[1]
1	0	[1]

| 1 | 1 | [1] |

SECTION B: 3-Input Truth Tables [8 marks]

4. Complete the truth table for $Q = (A \text{ AND } B) \text{ NOR } C$

A	B	C	A AND B	(A AND B) NOR C
0	0	0	[1]	[1]
0	0	1	[1]	[1]
0	1	0	[1]	[1]
0	1	1	[1]	[1]
1	0	0	[1]	[1]
1	0	1	[1]	[1]
1	1	0	[1]	[1]
1	1	1	[1]	[1]

5. Complete the truth table for $Q = A \text{ XOR } (B \text{ AND } C)$

A	B	C	B AND C	A XOR (B AND C)
0	0	0	[1]	[1]
0	0	1	[1]	[1]
0	1	0	[1]	[1]
0	1	1	[1]	[1]
1	0	0	[1]	[1]
1	0	1	[1]	[1]
1	1	0	[1]	[1]
1	1	1	[1]	[1]

SECTION C: Half-Adder [6 marks]

A half-adder adds two single bits. It has two outputs:

SUM = A XOR B

CARRY = A AND B

6. Complete the half-adder truth table:

A	B	SUM (A XOR B)	CARRY (A AND B)
0	0	[1]	[1]
0	1	[1]	[1]
1	0	[1]	[1]
1	1	[1]	[1]

7. Using the half-adder, what is $1 + 1$ in binary?

SUM = _____ CARRY = _____

So $1 + 1 =$ _____ (in binary) [2]

77. Explain why a half-adder is important in a computer's CPU.

_____ [2]

WORKSHEET 5B — NAND UNIVERSAL GATE & CIRCUIT PROBLEMS

Name: _____ Class: _____ Date: _____

Learning Objective: I can explain why NAND is a universal gate, build other gates from NAND, and convert between expressions, circuits, and truth tables.

Bilingual Support:

- universal gate = 通用门 (tong yong men)
- equivalent = 等效的 (deng xiao de)
- convert = 转换 (zhuan huan)

SECTION A: Build Gates from NAND [6 marks]

78. NOT gate from NAND: Draw a NAND gate with both inputs connected to A.

Complete the proof:

If $A = 0$: $0 \text{ NAND } 0 =$ _____ [1]

If $A = 1$: $1 \text{ NAND } 1 =$ _____ [1]

This matches NOT, so $Q = \text{NOT } A$ OK

79. AND gate from NAND: Draw two NAND gates to create an AND gate.

(Hint: NAND then NOT)

[Draw here]

[2 marks for correct diagram]

80. Explain in words why NAND is called a "universal gate." [2]

SECTION B: Convert Between Representations [9 marks]

4. Convert this expression to a circuit diagram:

$Q = (A \text{ AND } B) \text{ OR } (\text{NOT } C)$

[Draw circuit here — 3 marks]

5. Convert this circuit to a truth table:

A ---[OR]--- ---[AND]--- Q

B --- C ---

| A | B | C | A OR B | Q = (A OR B) AND C |

0 0 0	[1]		[1]	
0 0 1	[1]		[1]	
0 1 0	[1]		[1]	
0 1 1	[1]		[1]	
1 0 0	[1]		[1]	
1 0 1	[1]		[1]	

6. From this truth table, write the simplest expression:

A B Q
0 0 1
0 1 0
1 0 1
1 1 0

Expression: Q = _____ [3]

SECTION C: Real-World Circuit Design [5 marks]

7. A factory safety system has three sensors:

T = temperature normal (1 = normal, 0 = too high)

P = pressure normal (1 = normal, 0 = too high)

M = manual override pressed (1 = pressed, 0 = not pressed)

The safe light turns ON if:

(temperature is normal AND pressure is normal) OR manual override

(a) Write the logic expression: _____ [2]

(b) Draw the circuit diagram: [3]

[Draw here]

ANSWER KEY — WEEK 5

WORKSHEET 5A:

SECTION A:

1. NAND: 1, 1, 1, 0 [1 mark each = 4 marks]

2. NOR: 1, 0, 0, 0 [1 mark each = 4 marks]

3. XOR: 0, 1, 1, 0 [1 mark each = 4 marks]

SECTION B:

4. $Q = (A \text{ AND } B) \text{ NOR } C$:

A	B	C	A AND B	(A AND B) NOR C
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	0	0
1	0	0	0	1
1	0	1	0	0
1	1	0	1	0
1	1	1	1	0

5. $Q = A \text{ XOR } (B \text{ AND } C)$:

A	B	C	B AND C	A XOR (B AND C)
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

SECTION C:

6. Half-adder:

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

7. SUM = 0, CARRY = 1, so $1 + 1 = 10$ (binary) [2]

81. The half-adder is important because it is the building block for all arithmetic in a CPU [1]. By combining many half-adders (and full-adders), a computer can add any binary numbers [1]. Accept equivalent explanation.

WORKSHEET 5B:

SECTION A:

1. NOT from NAND:

If $A = 0$: $0 \text{ NAND } 0 = 1$ [1]

If $A = 1$: $1 \text{ NAND } 1 = 0$ [1]

82. AND from NAND: Two NAND gates. First NAND: A and $B \rightarrow \text{NOT}(A \text{ AND } B)$.

Second NAND: inputs joined $\rightarrow \text{NOT}(\text{NOT}(A \text{ AND } B)) = A \text{ AND } B$. [2]

Accept any correctly drawn equivalent circuit.

83. A universal gate can be used to create any other logic gate [1].

NAND is called universal because you can build NOT, AND, OR, and

therefore any other gate using only NAND gates [1].

SECTION B:

84. A and B to AND gate. C to NOT gate. AND output and NOT output to OR gate.

OR output is Q . [3]

(1 mark for correct gates, 1 mark for NOT on C , 1 mark for connections)

5. $Q = (A \text{ OR } B) \text{ AND } C$:

A	B	C	$A \text{ OR } B$	$Q = (A \text{ OR } B) \text{ AND } C$
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	1
1	0	0	1	0
1	0	1	1	1

6. $Q = A \text{ XOR } B$ (output is 1 when A and B are different) [3]

Accept " $Q = (A \text{ AND NOT } B) \text{ OR } (\text{NOT } A \text{ AND } B)$ " as equivalent [3]

SECTION C:

85. (a) $Q = (T \text{ AND } P) \text{ OR } M$ [2]

(b) T and P to AND gate. AND output and M to OR gate. OR output is Q .

Accept equivalent correct diagram. [3]

WEEK 6: SORTING ALGORITHMS — BUBBLE SORT

Overview: This week covers bubble sort — the simplest sorting algorithm

and the only one required by Cambridge IGCSE. Students learn the step-by-step

mechanic (comparing adjacent elements and swapping if out of order), trace through passes with physical cards and on paper, write bubble sort in pseudocode, implement it in Python, and analyse its efficiency. The key insight is that bubble sort makes multiple passes, with the largest element "bubbling" to the end each time. Students must understand best/average/worst case scenarios (all $O(n^2)$) and why bubble sort is rarely used in practice for large datasets.

Syllabus Ref: CIE 0478 Topic 2.1 (Algorithms — Bubble Sort)

Prerequisites:

- Y9S2: Arrays/lists, pseudocode, Python (loops, if statements, arrays)
- Y9S2: Linear search and binary search (understanding of searching vs sorting)

LESSON 6.1 — BUBBLE SORT: HOW IT WORKS

Topic: Bubble Sort Algorithm, Step-by-Step Tracing, Pseudocode, Number of Passes

Duration: 40 minutes

Lesson Type: Conceptual Introduction with Physical and Paper Tracing

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to describe how bubble sort works, trace through a bubble sort on paper, identify the number of passes needed for n elements, and write bubble sort in pseudocode.

Key Vocabulary:

- bubble sort = 冒泡排序 (mào pào pái xù)
- sorting = 排序 (pái xù)
- pass = 遍历/轮 (biàn lì / lún)
- swap = 交换 (jiāo huàn)
- compare = 比较 (bǐ jiào)
- adjacent = 相邻的 (xiāng lín de)
- sorted = 已排序 (yǐ pái xù)
- unsorted = 未排序 (wèi pái xù)
- algorithm = 算法 (suàn fǎ)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You have these numbers: [5, 2, 8, 1]. You need to arrange
- | them from smallest to largest. Describe your method."
- | Write steps in notebook. Think-Pair-Share.
- | (Collect methods — most will describe some form of comparison)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) What is sorting?
- | - Arranging data in a specific order (ascending or descending)
- | - Essential operation in computing:
- | - Phone contacts sorted alphabetically
- | - Exam results sorted by score
- | - Shopping websites sorted by price
- | - Many sorting algorithms exist; bubble sort is the simplest
- | (Slide 2) How Bubble Sort works:
- | Step 1: Compare the first two elements. If out of order, swap them.
- | Step 2: Move to the next pair. Compare, swap if needed.
- | Step 3: Continue to the end of the list.
- | Step 4: This completes ONE PASS. The largest element has "bubbled" to the end.
- | Step 5: Repeat passes, each time comparing one fewer pair (because the end is already sorted).
- | Step 6: When a complete pass makes NO swaps, the list is sorted.
- | (Slide 3) Worked Example — Sort [4, 2, 7, 1] ascending:
- | PASS 1:
- | Start: [4, 2, 7, 1]
- | Compare 4 and 2 -> swap -> [2, 4, 7, 1]
- | Compare 4 and 7 -> OK -> [2, 4, 7, 1]
- | Compare 7 and 1 -> swap -> [2, 4, 1, 7] <- 7 is in place!
- | PASS 2:
- | Start: [2, 4, 1, 7]
- | Compare 2 and 4 -> OK -> [2, 4, 1, 7]
- | Compare 4 and 1 -> swap -> [2, 1, 4, 7] <- 4 is in place!
- | PASS 3:
- | Start: [2, 1, 4, 7]
- | Compare 2 and 1 -> swap -> [1, 2, 4, 7]
- | PASS 4:
- | Start: [1, 2, 4, 7]
- | Compare 1 and 2 -> OK
- | No swaps made! List is SORTED.
- | (Slide 4) Key facts:

- | - After pass 1, the largest element is at the end
- | - After pass 2, the 2 largest elements are in place
- | - For n elements, maximum n-1 passes needed
- | - Bubble sort always compares ADJACENT elements only

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class with physical cards):
- | Give 5 students number cards: [3, 1, 4, 2, 5]
- | Students stand in line. Class guides them through bubble sort.
- | Each "compare and swap" requires students to physically move.
- | Count the number of swaps. Count the number of passes.
- | Exercise 2 (pairs, on paper):
- | Trace bubble sort on [6, 3, 1, 4]:
- | PASS 1:
- | [6, 3, 1, 4] -> compare 6,3 -> swap -> [3, 6, 1, 4]
- | [3, 6, 1, 4] -> compare 6,1 -> swap -> [3, 1, 6, 4]
- | [3, 1, 6, 4] -> compare 6,4 -> swap -> [3, 1, 4, 6]
- | PASS 2:
- | [3, 1, 4, 6] -> compare 3,1 -> swap -> [1, 3, 4, 6]
- | [1, 3, 4, 6] -> compare 3,4 -> OK
- | PASS 3:
- | [1, 3, 4, 6] -> compare 1,3 -> OK
- | No swaps! Done.
- | How many passes? 3 (but last pass was just a check)
- | How many swaps total? 4

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 6A — Bubble Sort Tracing & Pseudocode
- | Section A: Trace bubble sort on given arrays
- | Section B: Count passes and swaps
- | Section C: Write bubble sort pseudocode
- | (Worksheet 6A distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Bubble Sort Race"
- | Two students compete: each gets 5 number cards.
- | Student A uses bubble sort (swap adjacent only).
- | Student B can look at all cards and place them.
- | (B will always win — this sets up efficiency discussion!)

- | Discussion: "Why is bubble sort slow?"
- | - Can only move elements one position at a time
- | - Must make many passes even if nearly sorted
- | - Cannot "jump" elements like a human can
- | Homework: Trace bubble sort on [5, 4, 3, 2, 1] (reverse order).
- | Show every pass. Count total swaps. Count total comparisons.

EAL Support:

- "Bubble" metaphor: largest element "floats" to the top like a bubble in water (冒泡 = bubbling up)
- Physical card activity — kinaesthetic learning
- Sentence frame: "I compare ____ and _____. ____ > _____, so I swap them."
- Visual tracker: highlight sorted portion in green, unsorted in yellow
- Step-by-step trace template pre-printed

Differentiation:

Emerging | Use physical number cards for all tracing. Complete trace
 | with teacher guidance for a 3-element array only. Count
 | passes with visual help. Complete Section A only.

Developing | Trace bubble sort on paper for 4-5 element arrays.
 | Write pseudocode with scaffolding. Complete all sections.

Secure | Extension: Trace bubble sort on [1, 2, 3, 4, 5] (already
 | sorted). How many passes? What optimisation could we add?
 | (Answer: if no swaps in a pass, stop early). Research:
 | What is the difference between bubble sort and insertion sort?

Resources Needed:

- Number cards (0-9, laminated, multiple sets)
- Slide deck (bubble sort steps, worked example)
- Worksheet 6A (one per student)
- Trace template (pre-printed tables for passes)
- Whiteboard for class demonstration

LESSON 6.2 — BUBBLE SORT IN PYTHON & EFFICIENCY ANALYSIS

Topic: Python Implementation, Testing, Efficiency (Best/Average/Worst Case)

Duration: 40 minutes

Lesson Type: Practical Coding with Analysis

Syllabus Ref: CIE 0478 Topic 2.1

Learning Objective:

By the end of this lesson, all students will be able to implement bubble sort in Python, test it with different inputs, explain the difference between sorting and searching, and describe bubble sort's efficiency in best, average, and worst cases.

Key Vocabulary:

- implement = 实现 (shí xiàn)
- efficiency = 效率 (xiào lǜ)
- best case = 最好情况 (zuì hǎo qíng kuàng)
- worst case = 最坏情况 (zuì huài qíng kuàng)
- average case = 平均情况 (píng jūn qíng kuàng)
- time complexity = 时间复杂度 (shí jiān fù zá dù)
- Big O notation = 大 O 表示法 (dà O biǎo shì fǎ)
- already sorted = 已经排好序 (yǐ jīng pái hǎo xù)
- reverse order = 逆序 (nì xù)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "What is the difference between SEARCHING and SORTING?
- | Give an example of each from daily life."
- | Write in notebook. Share with partner.
- | (Searching = finding one item. Sorting = arranging all items.)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Searching vs Sorting:
 - | SEARCHING: Find ONE specific item in a collection
 - | - Linear search — check each item one by one
 - | - Binary search — divide and conquer (needs sorted data!)
 - | - Answer: "Is X in the list? Where is it?"
 - | SORTING: Arrange ALL items in a specific order
 - | - Bubble sort — compare adjacent, swap if needed
 - | - Many others: quick sort, merge sort, insertion sort
 - | - Answer: "Put everything in order"
- | Key point: Binary search NEEDS sorted data. Sorting enables efficient searching.
- | (Slide 2) Bubble Sort in Python:
 - | `def bubble_sort(arr):`
 - | `n = len(arr)`

```

|   for i in range(n):      <- pass number
|       for j in range(0, n-i-1): <- comparisons this pass
|           if arr[j] > arr[j+1]: <- compare adjacent
|               arr[j], arr[j+1] = arr[j+1], arr[j] <- swap
|   return arr
|
| # Test
| numbers = [64, 34, 25, 12, 22, 11, 90]
| print(bubble_sort(numbers))
| # Output: [11, 12, 22, 25, 34, 64, 90]

```

| (Slide 3) How the Python code works:

- | - Outer loop (i): controls the number of passes
- | - Inner loop (j): compares adjacent elements
- | - range(0, n-i-1): each pass compares fewer pairs
| (because the end is already sorted)
- | - The swap uses Python's multiple assignment

| (Slide 4) Efficiency Analysis:

- | BEST CASE: Array is already sorted
 - | -> n-1 comparisons, 0 swaps
 - | -> But standard bubble sort still does all passes!
 - | -> $O(n^2)$ unless optimised
- | WORST CASE: Array is in reverse order
 - | -> Maximum comparisons AND maximum swaps
 - | -> $O(n^2)$
- | AVERAGE CASE: Random order
 - | -> $O(n^2)$
- | Big O: Bubble sort is $O(n^2)$ in ALL cases (without optimisation)
- | This means: if data doubles, time quadruples!

| (Slide 5) Why learn bubble sort?

- | - Simple to understand and implement
- | - Required for Cambridge IGCSE
- | - Educational — helps understand sorting concepts
- | - NOT used in real applications for large data
- | - Better algorithms: quick sort $O(n \log n)$, merge sort $O(n \log n)$

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class in Thonny):
- | Type the bubble_sort function together.

```
| Test with: [5, 2, 8, 1, 9]
| Add print statements to show each pass:
| def bubble_sort_verbose(arr):
|     n = len(arr)
|     for i in range(n):
|         swapped = False
|         for j in range(0, n-i-1):
|             if arr[j] > arr[j+1]:
|                 arr[j], arr[j+1] = arr[j+1], arr[j]
|                 swapped = True
|         print(f"Pass {i+1}: {arr}")
|         if not swapped:
|             break # Optimisation!
|     return arr
| Exercise 2 (pairs):
| Test with already sorted array [1, 2, 3, 4, 5].
| How many passes with optimisation? Just 1!
| This is the BEST CASE with optimisation: O(n)
| Exercise 3 (individual):
| Modify the function to sort in DESCENDING order.
| (Hint: change the comparison from > to <)
```

25-35 min | INDEPENDENT PRACTICE

```
| Worksheet 6B — Bubble Sort in Python & Efficiency
| Section A: Trace through Python code execution
| Section B: Implement and test bubble sort in Thonny
| Section C: Efficiency analysis questions
| (Worksheet 6B distributed — see below)
```

35-40 min | PLENARY + HOMEWORK

```
| Plenary: "Efficiency Discussion"
| "If bubble sort takes 1 second for 1000 items, roughly
| how long for 10,000 items?"
| -> 100 times longer (10,000/1000 = 10, squared = 100)
| -> About 100 seconds (nearly 2 minutes!)
| "A better algorithm (O(n log n)) takes 1 second for 1000.
| How long for 10,000?"
| -> About 10-15 seconds (much faster!)
```

- | Homework: Implement bubble sort in Python to sort student
- | names alphabetically. Test with:
- | `names = ["Chen", "Wang", "Liu", "Zhang", "Li"]`
- | Does it work with strings? Why or why not? Explain.

EAL Support:

- Side-by-side pseudocode and Python comparison
- Colour-code: outer loop (blue), inner loop (green), swap (red)
- Annotated Python code with Chinese comments provided
- Sentence frames for efficiency explanation:
 - "In the best case, bubble sort is ____ because ____"
 - "In the worst case, bubble sort is ____ because ____"
- Visual graph showing n vs n^2 growth

Differentiation:

Emerging | Complete provided Python code with blanks to fill in.

- | Focus on tracing execution with print statements.
- | Answer efficiency questions using provided sentence frames.
- | Complete Sections A and B only.

Developing | Implement bubble sort from pseudocode independently.

- | Test with various inputs. Modify for descending order.
- | Explain efficiency in own words with examples.

Secure | Extension: Implement the "swapped" optimisation and

- | demonstrate it improves best case to $O(n)$. Compare
- | timing with and without optimisation using Python's
- | time module. Research: How does Python's built-in
- | `sort()` work? (Timsort — hybrid of merge and insertion)

Resources Needed:

- Thonny IDE on all computers
- Slide deck (Python code, efficiency graphs)
- Worksheet 6B (one per student)
- Annotated code handout
- Timer/stopwatch for optional timing experiments

WORKSHEET 6A — BUBBLE SORT TRACING & PSEUDOCODE

Name: _____ Class: _____ Date: _____

Learning Objective: I can trace bubble sort step-by-step and write it in pseudocode.

Bilingual Support:

- bubble sort = 冒泡排序 (mào pào pái xù)
- sort = 排序 (pái xù)
- pass = 轮 (lún)
- swap = 交换 (jiāo huàn)
- compare = 比较 (bǐ jiào)
- adjacent = 相邻的 (xiāng lín de)
- sorted = 已排序 (yǐ pái xù)

SECTION A: Trace Bubble Sort [8 marks]

1. Trace bubble sort on this array: [7, 3, 5, 1]

Show each comparison. Circle swaps.

PASS 1:

Start: [7, 3, 5, 1]

Compare 7 and 3 -> _____ -> [_____] [1]

Compare ____ and ____ -> _____ -> [_____] [1]

Compare ____ and ____ -> _____ -> [_____] [1]

PASS 2:

Start: [_____] [1]

Compare ____ and ____ -> _____ -> [_____] [1]

Compare ____ and ____ -> _____ -> [_____] [1]

PASS 3:

Start: [_____] [1]

Compare ____ and ____ -> _____ -> [_____] [1]

PASS 4:

Start: [_____] [1]

Any swaps? _____ [1]

Final sorted array: [_____] [1]

SECTION B: Count Passes and Swaps [4 marks]

2. For the array [4, 3, 2, 1] sorted with bubble sort:

Number of passes needed: _____ [1]

Number of swaps: _____ [1]

3. For the array [1, 2, 3, 4] sorted with bubble sort:

Number of passes needed: _____ [1]

Number of swaps: _____ [1]

86. Explain why the number of swaps is different for questions 2 and 3.

_____ [2]

SECTION C: Write Bubble Sort Pseudocode [8 marks]

5. Complete the pseudocode for bubble sort:

```
PROCEDURE BubbleSort(arr, n)
  FOR i FROM 0 TO n - 1          [1]
    FOR j FROM 0 TO _____    [2]
      IF arr[___] > arr[___ + 1] THEN [2]
        _____                [2]
        _____                [1]
      ENDIF
    ENDFOR
  ENDFOR
ENDPROCEDURE
```

87. Explain what the outer loop (i) controls in bubble sort. [2]

WORKSHEET 6B — BUBBLE SORT IN PYTHON & EFFICIENCY

Name: _____ Class: _____ Date: _____

Learning Objective: I can implement bubble sort in Python and explain its efficiency.

Bilingual Support:

- implement = 实现 (shí xiàn)
- efficiency = 效率 (xiào lǜ)
- best case = 最好情况 (zuì hǎo qíng kuàng)
- worst case = 最坏情况 (zuì huài qíng kuàng)
- Big O = 大 O (dà O)

SECTION A: Trace Python Code Execution [6 marks]

88. This Python code implements bubble sort. Trace what happens with
numbers = [3, 1, 4, 2]:

```
def bubble_sort(arr):
    n = len(arr)      # n = _____ [1]
    for i in range(n): # outer loop runs _____ times [1]
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
```



```
arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
return arr
```

When $i = 0$, j goes from 0 to _____. [1]

When $i = 1$, j goes from 0 to _____. [1]

When $i = 2$, j goes from 0 to _____. [1]

When $i = 3$, j goes from 0 to _____. [1]

SECTION B: Implement Bubble Sort [8 marks]

89. Type the bubble_sort function into Thonny. Test with these arrays and record the results:

Array 1: [64, 25, 12, 22, 11]

Result: _____ [2]

Array 2: [5, 4, 3, 2, 1]

Result: _____ [2]

Array 3: [1, 2, 3, 4, 5]

Result: _____ [2]

90. Modify the function to sort in DESCENDING order. Write your code:

_____ [2]

SECTION C: Efficiency Analysis [6 marks]

4. Circle the correct answer for each:

(a) Bubble sort best case time complexity:

$O(n)$ $O(n \log n)$ $O(n^2)$ [1]

(b) Bubble sort worst case time complexity:

$O(n)$ $O(n \log n)$ $O(n^2)$ [1]

(c) For 1000 items, bubble sort worst case does roughly:

1000 comparisons 500,000 comparisons 1,000,000 comparisons [1]

5. Explain why bubble sort is called $O(n^2)$. Use the words

"passes" and "comparisons" in your answer. [3]

ANSWER KEY — WEEK 6

WORKSHEET 6A:

SECTION A:

1. Trace bubble sort [7, 3, 5, 1]:

PASS 1:

Start: [7, 3, 5, 1]

Compare 7 and 3 -> swap -> [3, 7, 5, 1] [1]

Compare 7 and 5 -> swap -> [3, 5, 7, 1] [1]

Compare 7 and 1 -> swap -> [3, 5, 1, 7] [1]

PASS 2:

Start: [3, 5, 1, 7]

Compare 3 and 5 -> OK -> [3, 5, 1, 7] [1]

Compare 5 and 1 -> swap -> [3, 1, 5, 7] [1]

PASS 3:

Start: [3, 1, 5, 7]

Compare 3 and 1 -> swap -> [1, 3, 5, 7] [1]

PASS 4:

Start: [1, 3, 5, 7]

Any swaps? No swaps [1]

Final sorted array: [1, 3, 5, 7] [1]

SECTION B:

91. 4 elements -> 3 passes [1]. Swaps: 6 [1] ($3+2+1 = 6$)

92. 4 elements -> 3 passes (or 4 without optimisation) [1]. Swaps: 0 [1]

93. In question 2, the array is in reverse order, so EVERY comparison requires a swap [1]. In question 3, the array is already sorted, so no swaps are ever needed [1].

SECTION C:

5. Completed pseudocode:

PROCEDURE BubbleSort(arr, n)

FOR i FROM 0 TO n - 1 [1]

FOR j FROM 0 TO n - i - 2 [2]

IF arr[j] > arr[j + 1] THEN [2]

temp <- arr[j] [2]

arr[j] <- arr[j + 1] [1]

arr[j + 1] <- temp [1]

ENDIF

ENDFOR

ENDFOR

ENDPROCEDURE

Accept any logically correct swap method (e.g., using SWAP keyword,

or temp variable approach).

94. The outer loop controls the number of passes through the array [1].

Each pass places the next largest element in its correct position

at the end of the array [1].

WORKSHEET 6B:

SECTION A:

95. $n = 4$ [1]

Outer loop runs 4 times [1]

When $i = 0$, j goes from 0 to 2 [1]

When $i = 1$, j goes from 0 to 1 [1]

When $i = 2$, j goes from 0 to 0 [1]

When $i = 3$, j goes from 0 to -1 (loop doesn't execute) [1]

SECTION B:

2. Array 1: [11, 12, 22, 25, 64] [2]

Array 2: [1, 2, 3, 4, 5] [2]

Array 3: [1, 2, 3, 4, 5] (already sorted) [2]

3. Change comparison from $>$ to $<$:

```
def bubble_sort_descending(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] < arr[j+1]: # Changed > to <
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

[2] — Accept any correct modification.

SECTION C:

96. (a) $O(n^2)$ [1] — accept $O(n)$ if student notes optimisation

(b) $O(n^2)$ [1]

(c) 500,000 comparisons [1] — $n(n-1)/2 \sim n^2/2$

97. Bubble sort is $O(n^2)$ because for n elements, it makes up to $n-1$ passes [1],

and each pass makes up to $n-1$ comparisons [1]. This gives approximately

$n \times n = n^2$ comparisons in total [1]. Accept equivalent explanation.

WEEK 7: VALIDATION CHECKS

Overview: This week covers validation — ensuring that data entered into a program is sensible and within expected boundaries. Students learn five types of validation check (range, type, length, presence, format), understand that validation does NOT guarantee correctness (only that data is reasonable), and practise implementing validation in Python. Lesson 2 focuses on combining multiple checks and writing robust input routines that handle errors gracefully. All examples use culturally relevant contexts: student age validation, email format checking, Chinese phone number length validation.

Syllabus Ref: CIE 0478 Topic 2.2 (Validation)

Prerequisites:

- Y9S2: Python (if statements, loops, input/output, functions)
- Y9S2: Understanding of data types (integer, string, real)

LESSON 7.1 — TYPES OF VALIDATION CHECKS

Topic: Range Check, Type Check, Length Check, Presence Check, Format Check

Duration: 40 minutes

Lesson Type: Conceptual with Practical Python Coding

Syllabus Ref: CIE 0478 Topic 2.2

Learning Objective:

By the end of this lesson, all students will be able to explain five types of validation check, identify which check is appropriate for a given scenario, and implement range, type, length, presence, and format checks in Python.

Key Vocabulary:

- validation = 验证 (yàn zhèng)
- range check = 范围检查 (fàn wéi jiǎn chá)
- type check = 类型检查 (lèi xíng jiǎn chá)
- length check = 长度检查 (cháng dù jiǎn chá)
- presence check = 存在检查 (cún zài jiǎn chá)
- format check = 格式检查 (gé shì jiǎn chá)
- input = 输入 (shū rù)
- robust = 健壮的 (jiàn zhuàng de)
- verify = 核实 (hé shí)
- error message = 错误信息 (cuò wù xìn xī)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "A website asks for your age. You type 'abc'.
- | What should the program do? Why?"
- | Write answer in notebook. Share with partner.
- | (Expected: reject input, show error message, ask again)

3-12 min | **DIRECT INSTRUCTION**

- | (Slide 1) What is validation?
 - | - Checking that data entered is SENSIBLE and REASONABLE
 - | - Does NOT guarantee data is correct!
 - | Example: User enters age 150. It's a valid number
 - | (passes type check) but probably wrong (fails range check)
 - | - Does NOT guarantee data is accurate!
 - | Example: User enters age 25 but is actually 30.
 - | Validation cannot catch this — only verification can.
- | (Slide 2) Five Types of Validation Check:
 - | 1. RANGE CHECK — value must be between minimum and maximum
 - | Example: Age must be 11-18 (school student)
 - | Python: `if 11 <= age <= 18:`
 - | 2. TYPE CHECK — value must be the correct data type
 - | Example: Age must be a number, not text
 - | Python: `try: age = int(input("Age: "))`
 - | `except ValueError:`
 - | 3. LENGTH CHECK — value must have correct number of characters
 - | Example: Phone number must be 11 digits in China
 - | Python: `if len(phone) == 11:`
 - | 4. PRESENCE CHECK — value must not be empty
 - | Example: Name cannot be blank
 - | Python: `if name != "":`
 - | 5. FORMAT CHECK — value must match a specific pattern
 - | Example: Email must contain @ and .
 - | Python: `if "@" in email and "." in email:`
- | (Slide 3) Validation vs Verification:
 - | VALIDATION: "Is this data sensible?" (computer checks)
 - | VERIFICATION: "Is this data correct?" (human checks)
 - | Example: Enter password twice = verification
 - | Checking password is at least 8 characters = validation
- | (Slide 4) Validation in real-world apps:

- | - WeChat: phone number length check (11 digits)
- | - Taobao: password strength check (length + format)
- | - School system: age range check (11-18)
- | - Email: format check (@ symbol required)

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class in Thonny):
 - | Range check for student age:
 - | `age = int(input("Enter student age (11-18): "))`
 - | `if 11 <= age <= 18:`
 - | `print("Age valid")`
 - | `else:`
 - | `print("Age must be between 11 and 18")`
 - | Test with: 15 (valid), 10 (invalid), 20 (invalid)
- | Exercise 2 (pairs):
 - | Type check with error handling:
 - | `try:`
 - | `age = int(input("Enter age: "))`
 - | `print("Valid number")`
 - | `except ValueError:`
 - | `print("Please enter a whole number")`
 - | Test with: "15" (valid), "abc" (invalid), "15.5" (invalid)
- | Exercise 3 (individual):
 - | Format check for email:
 - | `email = input("Enter email: ")`
 - | `if "@" in email and "." in email:`
 - | `print("Email format valid")`
 - | `else:`
 - | `print("Email must contain @ and .")`
 - | Test with: "test@example.com" (valid),
 - | `"test.example.com" (invalid — no @),`
 - | `"test@example" (invalid — no .)`

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 7A — Validation Check Types
- | Section A: Identify the appropriate validation type for scenarios
- | Section B: Write Python code for each validation check
- | Section C: Explain validation vs verification

| (Worksheet 7A distributed — see below)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Validation Match Game"

| Teacher describes scenario, students call out check type:

| "Password cannot be empty" -> "Presence check!"

| "Postcode must be 6 digits" -> "Length check!"

| "Score must be 0-100" -> "Range check!"

| "Input must be a number" -> "Type check!"

| "Email needs @ and ." -> "Format check!"

| Homework: For each validation type, give a real-world example

| from an app or website you use. Explain what error message

| would be shown if validation fails.

EAL Support:

- Validation types poster on wall (bilingual with icons)

- Sentence frames:

"A ____ check is used when we need to check that ____"

"If the user enters ____, the program should ____ because ____"

- Colour-code: range (blue), type (green), length (orange),
presence (purple), format (red)

- Real app screenshots showing validation error messages (if available)

- Side-by-side Python code and explanation

Differentiation:

Emerging | Focus on identifying check types only (matching activity).

| Complete provided Python code with blanks. Use reference

| sheet for all check types. Complete Sections A and B only.

Developing | Write complete Python code for each check type independently.

| Identify appropriate checks for given scenarios. Complete

| all sections.

Secure | Extension: Research regular expressions (regex) in Python

| for advanced format checking. Write a function that

| validates a Chinese mobile phone number (starts with 1,

| followed by 3/4/5/7/8, total 11 digits). Discuss: What

| are the limitations of validation? Can you think of

| attacks that bypass validation?

Resources Needed:

- Slide deck (validation types, code examples)

- Thonny IDE for guided practice
- Validation types reference poster
- Worksheet 7A (one per student)
- Mini-whiteboards for plenary game

LESSON 7.2 — COMBINING VALIDATION CHECKS & ROBUST INPUT ROUTINES

Topic: Multiple Validation Checks, Loop Until Valid, Complete Input Validation

Duration: 40 minutes

Lesson Type: Practical Coding with Design Task

Syllabus Ref: CIE 0478 Topic 2.2

Learning Objective:

By the end of this lesson, all students will be able to combine multiple validation checks in a single input routine, use loops to repeatedly ask for input until it is valid, and write a robust form with comprehensive validation.

Key Vocabulary:

- combine = 组合 (zǔ hé)
- loop until valid = 循环直到有效 (xún huán zhí dào yǒu xiào)
- input routine = 输入例程 (shū rù lì chéng)
- retry = 重试 (chóng shì)
- comprehensive = 全面的 (quán miàn de)
- error handling = 错误处理 (cuò wù chǔ lǐ)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "You are writing a program to collect student information.
- | The user **MUST** enter a valid name, age (11-18), and email.
- | The program should keep asking until **ALL** inputs are valid.
- | Sketch the logic in your notebook."
- | Quick flowchart sketch. Think-Pair-Share.

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Combining validation checks:
- | Most real programs need **MULTIPLE** checks on the same input.
- | Example: Student age
- | Check 1: Presence (not empty)
- | Check 2: Type (must be integer)
- | Check 3: Range (must be 11-18)


```
| (Slide 2) Loop Until Valid:
|   Use a WHILE loop to keep asking until input passes ALL checks.
|   valid = False
|   while not valid:
|       age = input("Enter age: ")
|       if age == "":
|           print("Age cannot be empty")
|       elif not age.isdigit():
|           print("Age must be a number")
|       elif not (11 <= int(age) <= 18):
|           print("Age must be 11-18")
|       else:
|           valid = True
|           age = int(age)
| (Slide 3) A robust input function:
|   def get_valid_age():
|       while True:
|           try:
|               age = int(input("Enter student age (11-18): "))
|               if 11 <= age <= 18:
|                   return age
|           except:
|               print("Age must be between 11 and 18.")
|       except ValueError:
|           print("Please enter a whole number.")
|   # Use the function
|   student_age = get_valid_age()
| (Slide 4) Complete form with multiple fields:
|   def student_registration_form():
|       # Name — presence + length check
|       name = input("Enter full name: ").strip()
|       while name == "" or len(name) < 2:
|           print("Name must be at least 2 characters.")
|           name = input("Enter full name: ").strip()
|       # Age — type + range check
|       age = get_valid_age()
```

```

| # Email — format check
| email = input("Enter email: ").strip()
| while "@" not in email or "." not in email:
|     print("Invalid email format.")
|     email = input("Enter email: ").strip()
| print(f"Registration complete for {name}")
| return name, age, email

```

12-25 min | **GUIDED PRACTICE**

```

| Exercise 1 (whole class):
| Build a robust function for Chinese mobile number:
| def get_valid_phone():
|     while True:
|         phone = input("Enter 11-digit mobile: ").strip()
|         if len(phone) != 11:
|             print("Must be exactly 11 digits.")
|         elif not phone.isdigit():
|             print("Must contain only numbers.")
|         elif phone[0] != '1':
|             print("Must start with 1.")
|         elif phone[1] not in '34578':
|             print("Second digit must be 3,4,5,7, or 8.")
|         else:
|             return phone

```

```

| Exercise 2 (pairs):
| Write a function get_valid_score() that:

```

- | - Checks presence (not empty)
- | - Checks type (integer)
- | - Checks range (0-100)
- | - Returns the valid score

```

| Exercise 3 (individual):
| Add format check: exam paper code must be "P1" or "P2"
| (presence + format check)

```

25-35 min | **INDEPENDENT PRACTICE**

- | Worksheet 7B — Combining Validation & Robust Routines
- | Section A: Write combined validation functions
- | Section B: Complete form with multiple validated fields

- | Section C: Error analysis — find mistakes in validation code
- | (Worksheet 7B distributed — see below)

35-40 min | **PLENARY + HOMEWORK**

- | Plenary: "Code Review"
- | Display a student's validation code (with permission) or teacher-written example. Class reviews:
 - | - Which checks are included?
 - | - What happens if user enters wrong type?
 - | - Is the error message helpful?
 - | - What could be improved?
- | Homework: Write a complete student registration program with:
 - | - Name (presence + length check, minimum 2 characters)
 - | - Age (type + range check, 11-18)
 - | - Class (format check, must be "10A", "10B", or "10C")
 - | - Email (format check, must contain @ and .)
 - | Use a loop for each field until valid input is received.

EAL Support:

- Complete annotated code with Chinese comments
- Sentence frames for combining checks:
 - "First, I check for _____. If that passes, I check for _____."
 - "I use a while loop to keep asking until _____"
- Flowchart template showing validation logic visually
- Common Python error messages bilingual reference
- Highlight the try/except structure with colour-coding

Differentiation:

Emerging | Complete partially-written functions with blanks to fill in.

- | Use provided function templates. Focus on one validation type at a time. Complete Sections A and C only.

Developing | Write complete validation functions independently. Combine

- | 2-3 checks per input field. Build a form with 3+ fields.
- | Complete all sections.

Secure | Extension: Write a validation module with reusable functions

- | for each check type. Use dictionaries to store validation rules. Research: What is input sanitisation? How does it differ from validation? Write about SQL injection as an example of why validation matters for security.

Resources Needed:

- Thonny IDE on all computers
- Slide deck (combined validation, loop structures)
- Worksheet 7B (one per student)
- Annotated code handout
- Validation flowchart template

WORKSHEET 7A — VALIDATION CHECK TYPES

Name: _____ Class: _____ Date: _____

Learning Objective: I can identify and implement five types of validation check in Python.

Bilingual Support:

- validation = 验证 (yan zheng)
- range check = 范围检查 (fan wei jian cha)
- type check = 类型检查 (lei xing jian cha)
- length check = 长度检查 (chang du jian cha)
- presence check = 存在检查 (cun zai jian cha)
- format check = 格式检查 (ge shi jian cha)
- robust = 健壮的 (jian zhuang de)

SECTION A: Identify the Validation Type [5 marks]

For each scenario, write the name of the most appropriate validation check.

1. A password must be at least 8 characters long.

_____ [1]

2. An exam score must be between 0 and 100.

_____ [1]

98. A user must enter something in the name field (cannot leave blank).

_____ [1]

4. A phone number must contain only digits.

_____ [1]

5. An email address must contain @ and a domain name.

_____ [1]

SECTION B: Write Python Validation Code [15 marks]

Write Python code for each validation check. Include an appropriate error message.

6. Range check: Age must be 16-65.

_____ [3]

7. Type check: The input must be an integer.

(Use try/except)

_____ [3]

8. Length check: A username must be 5-15 characters.

_____ [3]

9. Presence check: The user must enter a value.

_____ [3]

99. Format check: A product code must start with "PR" followed by 4 digits.

_____ [3]

SECTION C: Validation vs Verification [5 marks]

100. Explain the difference between validation and verification.

Give an example of each. [4]

101. Can validation guarantee that data is correct? Explain. [1]

WORKSHEET 7B — COMBINING VALIDATION & ROBUST INPUT ROUTINES

Name: _____ Class: _____ Date: _____

Learning Objective: I can combine multiple validation checks and write robust input routines using loops.

Bilingual Support:

- combine = 组合 (zu he)

- loop until valid = 循环直到有效 (xun huan zhi dao you xiao)

- input routine = 输入例程 (shu ru li cheng)

- error handling = 错误处理 (cuo wu chu li)

SECTION A: Write Combined Validation Functions [9 marks]

Write a function for each scenario that uses a loop to keep asking until the input is valid.

102. def get_valid_age():

"""Gets a valid student age (11-18). Keeps asking until valid."""

_____ [3]

```
103. def get_valid_email():  
    """Gets a valid email (contains @ and .). Keeps asking until valid."""
```

_____ [3]

```
104. def get_valid_phone():  
    """Gets a valid Chinese mobile (11 digits, starts with 1)."""
```

_____ [3]

SECTION B: Complete Registration Form [6 marks]

105. Complete this student registration program with full validation:

```
def register_student():  
    # Get name — presence check + length check (at least 2 chars)  
    name = input("Enter name: ").strip()  
    while _____:  
        print("Name must be at least 2 characters.")  
        name = input("Enter name: ").strip()  
    # Get age — use your function from Q1  
    age = _____ [1]  
    # Get class — format check (must be 10A, 10B, or 10C)  
    class_name = input("Enter class (10A/10B/10C): ").strip()  
    while class_name not in _____:  
        print("Class must be 10A, 10B, or 10C.")  
        class_name = input("Enter class: ").strip()  
    print(f"Registered: {name}, Age {age}, Class {class_name}")  
    return name, age, class_name
```

[5 marks total for all blanks]

SECTION C: Find the Errors [5 marks]

106. This validation code has THREE errors. Find and correct them.

```
def get_score():  
    score = input("Enter score (0-100): ")  
    if score < 0 or score > 100:      # Error 1: _____ [1]  
        print("Score out of range")  
    elif type(score) != int:          # Error 2: _____ [1]  
        print("Must be a number")  
    else:  
        return score                 # Error 3: _____ [1]
```

Corrected code:

ANSWER KEY — WEEK 7

WORKSHEET 7A:

SECTION A:

1. Length check [1]**2. Range check [1]****3. Presence check [1]****4. Type check [1] — accept format check****5. Format check [1]**

SECTION B:

6. Range check for age 16-65:

```
age = int(input("Enter age (16-65): "))
```

```
if 16 <= age <= 65:
```

```
    print("Age valid")
```

```
else:
```

```
    print("Age must be between 16 and 65")
```

[3] — 1 for input, 1 for correct range condition, 1 for error message

7. Type check with try/except:

```
try:
```

```
    number = int(input("Enter a whole number: "))
```

```
    print("Valid integer")
```

```
except ValueError:
```

```
    print("Please enter a whole number")
```

[3] — 1 for try/except structure, 1 for int(), 1 for error message

8. Length check for username:

```
username = input("Enter username: ")
```

```
if 5 <= len(username) <= 15:
```

```

    print("Username valid")
else:
    print("Username must be 5-15 characters")
[3] — 1 for len(), 1 for correct condition, 1 for message

```

9. Presence check:

```

value = input("Enter a value: ").strip()
if value == "":
    print("This field cannot be empty")
else:
    print("Value accepted")
[3] — 1 for strip(), 1 for empty string check, 1 for message

```

10. Format check for product code:

```

code = input("Enter product code: ")
if code[:2] == "PR" and code[2:].isdigit() and len(code) == 6:
    print("Valid code")
else:
    print("Code must be PR followed by 4 digits")
[3] — 1 for checking "PR", 1 for checking digits, 1 for total length
Accept equivalent approaches (e.g., using startswith()).

```

SECTION C:

107. Validation checks that data is sensible/reasonable (e.g., age is a number within an expected range) [1]. Verification checks that data is correct/accurate (e.g., entering password twice to confirm it was typed correctly) [1]. Example of validation: checking that an exam score is between 0 and 100 [1]. Example of verification: asking a user to retype their email address to confirm it [1].
108. No, validation cannot guarantee correctness [1]. A user could enter age 25 when they are actually 30 — the value passes validation (it's a reasonable age) but it is not correct. Accept equivalent.

WORKSHEET 7B:

SECTION A:

109. `def get_valid_age():`
 `while True:`
 `try:`


```

    age = int(input("Enter age (11-18): "))
    if 11 <= age <= 18:
        return age
    print("Age must be 11-18.")
except ValueError:
    print("Please enter a whole number.")

```

[3] — 1 for loop structure, 1 for range check, 1 for type check

```

110. def get_valid_email():
    while True:
        email = input("Enter email: ").strip()
        if "@" in email and "." in email:
            return email
        print("Email must contain @ and .")

```

[3] — 1 for loop, 1 for format check, 1 for return

```

111. def get_valid_phone():
    while True:
        phone = input("Enter mobile: ").strip()
        if len(phone) == 11 and phone.isdigit() and phone[0] == '1':
            return phone
        print("Must be 11 digits starting with 1.")

```

[3] — 1 for loop, 1 for checks, 1 for return

SECTION B:

4. Completed register_student():

```

while len(name) < 2 or name == "": [1]
age = get_valid_age() [1]
while class_name not in ["10A", "10B", "10C"]: [1]
(1 mark each for other correct blanks, total 5)
Accept equivalent conditions.

```

SECTION C:

```

112. Error 1: score is still a string from input() — need int() [1]
Error 2: type() check should be try/except ValueError [1]
Error 3: return should return int(score), not string [1]
Corrected code:
def get_score():
    while True:

```

```

try:
    score = int(input("Enter score (0-100): "))
    if 0 <= score <= 100:
        return score
    print("Score out of range")
except ValueError:
    print("Must be a whole number")

```

[2] — Accept any logically correct rewrite. Award marks for fixing all three identified errors.

WEEK 8: NETWORK HARDWARE

Overview: This week provides in-depth coverage of network hardware — a topic briefly touched in Year 9 but now explored with the detail required for IGCSE. Students learn about NICs, MAC addresses, IP addresses (IPv4/IPv6), routers, switches, and network topologies. They compare client-server and peer-to-peer networks and apply their knowledge by designing a network diagram for VPA Shenzhen's computer lab. The content connects to students' daily experience of Wi-Fi, Ethernet, and internet connectivity.

Syllabus Ref: CIE 0478 Topic 3.3, 3.4

Prerequisites:

- Y9S1: Basic network concepts (LAN, WAN, internet)
- Y9S1: Brief introduction to network hardware (router, switch mentioned)

LESSON 8.1 — NETWORK HARDWARE: NIC, MAC, IP, ROUTERS, SWITCHES & TOPOLOGIES

Topic: NIC, MAC Address, IP Address (IPv4/IPv6), Routers, Switches, Network Topologies

Duration: 40 minutes

Lesson Type: Conceptual with Practical Identification and Design

Syllabus Ref: CIE 0478 Topic 3.3, 3.4

Learning Objective:

By the end of this lesson, all students will be able to explain the purpose and function of NICs, MAC addresses, IP addresses, routers, and switches, describe star, bus, and mesh topologies, and identify network hardware in their immediate environment.

Key Vocabulary:

- Network Interface Card (NIC) = 网络接口卡 (wang luo jie kou ka)
- MAC address = MAC 地址 (MAC di zhi) — Media Access Control address
- IP address = IP 地址 (IP di zhi) — Internet Protocol address

- router = 路由器 (lu you qi)
- switch = 交换机 (jiao huan ji)
- topology = 拓扑结构 (tuo pu jie gou)
- star topology = 星型拓扑 (xing xing tuo pu)
- bus topology = 总线型拓扑 (zong xian xing tuo pu)
- mesh topology = 网状拓扑 (wang zhuang tuo pu)
- packet = 数据包 (shu ju bao)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Look around the classroom. What physical network equipment can you see? How does your computer connect to the internet?"
- | Write observations. Think-Pair-Share.
- | (Expected: Wi-Fi router, ethernet cables, network ports)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) How devices connect to networks:
 - | Every networked device needs a NETWORK INTERFACE CARD (NIC).
 - | - Also called a network adapter
 - | - Can be wired (Ethernet port) or wireless (Wi-Fi adapter)
 - | - Every NIC has a unique MAC ADDRESS (burned into hardware)
- | (Slide 2) MAC Address:
 - | - Media Access Control address
 - | - A unique identifier assigned to every NIC by the manufacturer
 - | - Format: 6 pairs of hexadecimal digits, separated by colons
 - | Example: 00:1A:2B:3C:4D:5E
 - | - NEVER changes — it is permanent (unless spoofed)
 - | - Used for communication WITHIN a local network
 - | - Think of it as the device's "factory serial number"
- | (Slide 3) IP Address:
 - | - Internet Protocol address
 - | - A unique address assigned to a device when it joins a network
 - | - CAN change (unlike MAC address)
 - | - IPv4 format: 4 decimal numbers 0-255, separated by dots
 - | Example: 192.168.1.105
 - | - IPv6 format: 8 groups of 4 hex digits (newer, more addresses)
 - | Example: 2001:0DB8:AC10:FE01::0000
 - | - Used for communication BETWEEN networks (internet)

| (Slide 4) MAC vs IP:

+-----+-----+-----+			
		MAC Address	IP Address
+-----+-----+-----+			
	Format	Hex (00:1A:2B:...)	IPv4: 192.168.x.x
	Changes?	Never (permanent)	Yes (when moving)
	Scope	Local network	Global (internet)
	Assigned by	Manufacturer	Network/DHCP
	Purpose	Identify device	Locate device
+-----+-----+-----+			

| (Slide 5) Router:

- | - Connects different networks together
- | - Routes packets to their destination using IP addresses
- | - Your home router connects your LAN to the internet (WAN)
- | - Has a routing table to decide where to send packets
- | - Can have firewall functions built in

| (Slide 6) Switch:

- | - Connects devices WITHIN the same local network (LAN)
- | - Uses MAC addresses to forward data to the correct device
- | - More intelligent than a hub (which broadcasts to all)
- | - Each device connects to a port on the switch

| (Slide 7) Network Topologies:

| STAR: All devices connect to a central switch/hub

- | + Easy to add/remove devices
- | + If one cable fails, only one device affected
- | - Central device fails = whole network down
- | - Lots of cable needed

| BUS: All devices connect to one main cable (backbone)

- | + Less cable needed
- | + Easy to set up
- | - Backbone fails = whole network down
- | - Difficult to add more devices

| MESH: Every device connects to every other device

- | + Very reliable — multiple paths available
- | - Very expensive — lots of cables needed
- | - Complex to set up

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class):
 - | "What is the MAC address of this computer?"
 - | (Teacher demonstrates: ipconfig /all on Windows or ifconfig on Mac/Linux)
 - | Students identify their MAC address format.
- | Exercise 2 (pairs):
 - | "Your home has IP address 192.168.1.5. Your friend's home has 192.168.1.5. Can this work? Why or why not?"
 - | -> Yes! Because these are private IP addresses used inside different local networks. The router translates them.
- | Exercise 3 (individual):
 - | "Draw a star topology for 4 computers and a printer connected to a switch. Label all devices."

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 8A — Network Hardware & Topologies
- | Section A: Identify network hardware from descriptions
- | Section B: MAC vs IP address comparison
- | Section C: Draw and evaluate network topologies
- | (Worksheet 8A distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Network Hardware Quiz"
- | Quick-fire questions:
 - | "What device uses MAC addresses to forward data?" -> "Switch!"
 - | "What device uses IP addresses to route between networks?" -> "Router!"
 - | "Which address never changes?" -> "MAC address!"
 - | "Which topology has a central device?" -> "Star!"
 - | "Which topology is most reliable?" -> "Mesh!"
- | Homework: Draw and label the network topology of your home.
- | Identify: router, switch (if any), connected devices,
- | Wi-Fi vs wired connections.

EAL Support:

- Network hardware poster with photos of real equipment (bilingual labels)
- MAC vs IP comparison table on desk reference card
- Topology diagrams with pros/cons colour-coded (green=advantage, red=disadvantage)
- Sentence frames:

"A ____ uses ____ addresses to ____"

"In a ____ topology, if ____ fails, then ____"

- Real equipment to examine if available (old router, switch, NIC)

Differentiation:

Emerging | Match hardware names to pictures. Complete provided tables

| with word bank. Draw simple star topology only. Complete

| Section A and part of Section C only.

Developing | Complete all worksheet sections. Explain MAC vs IP in

| own words. Draw and evaluate all three topologies.

Secure | Extension: Research how DHCP assigns IP addresses. Explain

| NAT (Network Address Translation) and why private IP

| ranges exist. Design a hybrid topology for a multi-floor

| school building. Research: What is a VLAN?

Resources Needed:

- Slide deck (hardware diagrams, topology illustrations)

- Network hardware poster with real photos

- MAC vs IP reference cards

- Worksheet 8A (one per student)

- Whiteboard for topology drawing

- Optional: real router/switch for demonstration

LESSON 8.2 — CLIENT-SERVER vs PEER-TO-PEER & NETWORK DESIGN

Topic: Client-Server Networks, Peer-to-Peer Networks, Network Security,
Practical Network Design

Duration: 40 minutes

Lesson Type: Comparison and Practical Design Task

Syllabus Ref: CIE 0478 Topic 3.4

Learning Objective:

By the end of this lesson, all students will be able to explain the difference between client-server and peer-to-peer networks, describe basic network security measures (firewall), and design a network diagram for a given scenario.

Key Vocabulary:

- client-server = 客户端-服务器 (ke hu duan - fu wu qi)

- peer-to-peer (P2P) = 点对点 (dian dui dian)

- server = 服务器 (fu wu qi)

- client = 客户端 (ke hu duan)
- firewall = 防火墙 (fang huo qiang)
- centralised = 集中式的 (ji zhong shi de)
- decentralised = 分布式的 (fen bu shi de)
- network diagram = 网络图 (wang luo tu)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "When you use WeChat to send a message, where does the
- | message go? Does it go directly to your friend's phone,
- | or through a server?"
- | Write answer. Think-Pair-Share.
- | (Through Tencent's servers — client-server model)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Client-Server Network:
- | - One or more powerful computers called SERVERS
- | - Many client devices (computers, phones, tablets)
- | - Clients REQUEST services; SERVERS provide them
- | - All data flows through the central server
- | Diagram:
- | +-----+
- | | Server |
- | | (data) |
- | +---+---+
- | +-----+
- | +---+---+ +---+---+ +---+---+
- | | PC1 | | PC2 | | PC3 |
- | +-----+ +-----+ +-----+
- | Examples:
- | - School network — file server stores all student work
- | - Websites — web server delivers pages
- | - WeChat — message servers store and forward messages
- | - Online gaming — game server manages all players
- | Advantages:
- | + Central control — admin manages all data
- | + Security — data protected on server
- | + Backups — server backed up regularly

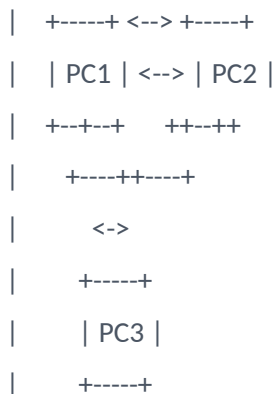
| Disadvantages:

- | - Server fails = whole network stops
- | - Expensive — powerful servers cost money
- | - Needs specialist staff to maintain

| (Slide 2) Peer-to-Peer (P2P) Network:

- | - No central server
- | - All devices are EQUAL — every device is client AND server
- | - Devices connect directly to each other

| Diagram:



| Examples:

- | - Bluetooth file sharing
- | - BitTorrent file sharing
- | - Early versions of AirDrop

| Advantages:

- | + Cheap — no server needed
- | + Easy to set up
- | + If one device fails, others still work

| Disadvantages:

- | - No central control or backup
- | - Security — each device manages its own security
- | - Performance slows as more devices join

| (Slide 3) Network Security — Firewall:

- | - A firewall filters network traffic
- | - Allows safe traffic, blocks dangerous traffic
- | - Can be hardware (built into router) or software
- | - Protects the network from unauthorised access

| What a firewall does:

- | + Block traffic from known bad IP addresses

- | + Prevent unauthorised access to internal network
- | + Monitor and log network activity
- | (Slide 4) Choosing the right network model:
- | School/Office -> Client-Server (needs control, security)
- | Small home -> Either (client-server for internet,
- | P2P for file sharing)
- | File sharing -> P2P (BitTorrent, decentralised)

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class):
- | "VPA Shenzhen has 200 students, 30 teachers, computer labs,
- | and a file server. Should they use client-server or P2P?"
- | -> Client-server: needs central file storage, user accounts,
- | security, backups.
- | Exercise 2 (pairs):
- | "Two friends want to share photos directly using Bluetooth.
- | What network model is this? What are the advantages?"
- | -> P2P: simple, no internet needed, direct connection.
- | Exercise 3 (individual):
- | "A school wants to block students from accessing certain
- | websites. Where should the firewall be placed?"
- | -> Between the school network and the internet (at the router).

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 8B — Client-Server, P2P & Network Design
- | Section A: Compare client-server and P2P (table completion)
- | Section B: Identify appropriate network model for scenarios
- | Section C: Design a network diagram for VPA Shenzhen
- | (Worksheet 8B distributed — see below)

35-40 min | PLENARY + HOMEWORK

- | Plenary: "Network Design Pitch"
- | Students present their VPA network diagram to a partner.
- | Partner asks: "Why did you choose this topology?"
- | "Where is the firewall?"
- | "What happens if the server fails?"
- | Homework: Research: "What is a Wi-Fi access point and how
- | does it differ from a router?" Write 3-4 sentences explaining.
- | Draw how Wi-Fi fits into your home network diagram from Lesson 1.

EAL Support:

- Client-server vs P2P comparison table (bilingual) on desk
- Sentence frames:
"In a client-server network, the ____ provides ____ to the ____."
"In a P2P network, every device is both a ____ and a ____."
- Visual diagram showing data flow in each model
- Colour-code: server (red), clients (blue), P2P peers (green)
- Real-world examples from Chinese apps (WeChat = client-server, Xender/SHAREit = P2P)

Differentiation:

Emerging | Complete comparison table with word bank. Match scenarios
| to network models. Draw simple network diagram with
| provided labels. Complete Sections A and B only.

Developing | Complete all sections. Explain advantages/disadvantages
| in own words. Design network diagram independently with
| justification for choices.

Secure | Extension: Design a network for a hypothetical 3-floor
| office building with 500 employees. Include: server room,
| wireless access points on each floor, firewall, backup
| internet connection. Research redundancy and failover
| systems. Explain VLAN segmentation.

Resources Needed:

- Slide deck (network models, diagrams, firewall explanation)
- Client-server vs P2P comparison handout
- Worksheet 8B (one per student)
- A3 paper for network diagrams
- Coloured pens for diagram drawing
- Rulers for neat connections

WORKSHEET 8A — NETWORK HARDWARE & TOPOLOGIES

Name: _____ Class: _____ Date: _____

Learning Objective: I can identify network hardware and describe network topologies.

Bilingual Support:

- NIC = 网络接口卡 (wang luo jie kou ka)
- MAC address = MAC 地址 (MAC di zhi)

- IP address = IP 地址 (IP di zhi)
- router = 路由器 (lu you qi)
- switch = 交换机 (jiao huan ji)
- topology = 拓扑结构 (tuo pu jie gou)

SECTION A: Identify the Hardware [5 marks]

Match each description to the correct hardware device.

113. A card or chip that allows a device to connect to a network _____ [1]
 114. A unique hardware address burned into every network adapter _____ [1]
 115. A device that forwards data using MAC addresses within a LAN _____ [1]
 116. A device that routes packets between networks using IP addresses _____ [1]
 117. A numerical address assigned to a device when it joins a network _____ [1]
- a) Router b) Switch c) NIC d) MAC address e) IP address

SECTION B: MAC vs IP Address [8 marks]

6. Complete this comparison table:

Feature	MAC Address	IP Address	
Format	_____	_____	[2]
Changes?	_____	_____	[2]
Assigned by	_____	_____	[2]
Used for	_____	_____	[2]

118. A computer has MAC address 00:1A:2B:3C:4D:5E and IP address 192.168.1.10.

When this computer moves to a different network:

- (a) What happens to its MAC address? _____ [1]
- (b) What happens to its IP address? _____ [1]

SECTION C: Network Topologies [7 marks]

119. For each topology, list ONE advantage and ONE disadvantage.

(a) Star topology:

Advantage: _____ [1]

Disadvantage: _____ [1]

(b) Bus topology:

Advantage: _____ [1]

Disadvantage: _____ [1]

(c) Mesh topology:

Advantage: _____ [1]

Disadvantage: _____ [1]

9. Draw a star topology with 1 switch and 4 computers. [1]

[Draw here]

WORKSHEET 8B — CLIENT-SERVER, P2P & NETWORK DESIGN

Name: _____ Class: _____ Date: _____

Learning Objective: I can compare client-server and P2P networks and design a network for a given scenario.

Bilingual Support:

- client-server = 客户端-服务器 (ke hu duan - fu wu qi)
- peer-to-peer = 点对点 (dian dui dian)
- server = 服务器 (fu wu qi)
- client = 客户端 (ke hu duan)
- firewall = 防火墙 (fang huo qiang)
- centralised = 集中式 (ji zhong shi)

SECTION A: Compare Client-Server and P2P [8 marks]

1. Complete the comparison table:

Feature	Client-Server	Peer-to-Peer	
Central server?	_____	_____	[1]
Device roles	_____	_____	[1]
Cost to set up	_____	_____	[1]
If one device fails	_____	_____	[1]
Security control	_____	_____	[1]
Example	_____	_____	[1]

120. Give TWO situations where client-server is better than P2P. [2]

SECTION B: Choose the Right Network Model [4 marks]

For each scenario, write "Client-Server" or "P2P" and explain why.

121. A school with 500 students needs to store all files centrally, manage user accounts, and run regular backups.

Model: _____ [1]

Reason: _____ [1]

122. Two friends want to share a large video file quickly without using mobile data.

Model: _____ [1]

Reason: _____ [1]

SECTION C: Design a Network for VPA Shenzhen [8 marks]

5. VPA Shenzhen has:

- Computer Lab A: 30 student computers, 1 teacher computer, 1 printer
- Computer Lab B: 30 student computers, 1 teacher computer, 1 printer
- Staff room: 10 teacher computers, 1 printer
- 1 school file server
- Internet connection via a router

(a) Draw a network diagram showing all devices, the server, router, and how they connect. Use a star topology. [4]

[Draw here]

(b) Label the following on your diagram: [2]

- Switch positions
- Server location
- Firewall location

(c) Explain ONE advantage of using client-server for this network. [1]

(d) What would happen if the main switch in Lab A failed? [1]

ANSWER KEY — WEEK 8

WORKSHEET 8A:

SECTION A:

123. c) NIC [1]
124. d) MAC address [1]
125. b) Switch [1]
126. a) Router [1]
127. e) IP address [1]

SECTION B:

6. MAC vs IP comparison:

Feature	MAC Address	IP Address	
Format	6 hex pairs (00:1A:2B)	IPv4: 4 decimals	
	(192.168.1.1)		
Changes?	Never (permanent)	Yes (can change)	
Assigned by	Manufacturer	Network/DHCP	
Used for	Communication within LAN	Communication	
	between networks		

[2 marks for format, 2 for changes, 2 for assigned by, 2 for used for]

Accept equivalent correct answers.

128. (a) MAC address stays the same [1] (it is permanent hardware address)
 (b) IP address will change [1] (new network assigns new IP)

SECTION C:

129. (a) Star: Advantage — easy to add/remove devices OR if one cable fails only one device affected [1]. Disadvantage — if central switch fails, whole network down OR expensive (lots of cable) [1].
 (b) Bus: Advantage — less cable needed OR easy to set up [1].
 Disadvantage — if backbone fails, whole network down OR difficult to add devices [1].
 (c) Mesh: Advantage — very reliable (multiple paths) OR no single point of failure [1]. Disadvantage — very expensive (lots of cables) OR complex to set up [1].
130. Star topology drawing: central switch with 4 computers connected by lines. [1] Accept any reasonable representation.

WORKSHEET 8B:

SECTION A:

1. Comparison table:

Feature	Client-Server	Peer-to-Peer	
Central server?	Yes	No	[1]
Device roles	Clients request, server provides	All devices are equal (client+server)	[1]
Cost to set up	Expensive (need server)	Cheap (no server)	[1]
If one device fails	Only that device affected (if not server)	Only that device affected	[1]
Security control	Central (server manages)	Each device manages its own security	[1]
Example	School network, WeChat	Bluetooth, BitTorrent	[1]

131. Any two valid: needs central data storage / needs user account management / needs regular backups / needs security control [1 each]

SECTION B:

132. Client-Server [1]. Reason: Central file storage, user account management, and backups require a central server [1]. Accept equivalent explanation.
133. P2P [1]. Reason: Direct connection without internet means no data charges and faster transfer [1]. Accept equivalent.

SECTION C:

134. (a) Network diagram: [4]

- Star topology shown [1]
- Two labs each with switch and computers shown [1]
- Staff room with switch and computers shown [1]
- Router, server, and internet connection shown [1]

Accept any logically correct diagram.

(b) Labels correct [2]

(c) Any valid advantage: central file storage accessible from any computer / user accounts managed centrally / regular backups of all files / security controlled centrally [1]

(d) If main switch in Lab A failed, all 30 student computers and the teacher computer in Lab A would lose network connection [1].

Other labs and staff room would still work [1].

WEEK 9: PAPER 1 REVIEW — DATA REPRESENTATION

Overview: This week consolidates ALL Data Representation content from Year 9 in preparation for IGCSE Paper 1. Lesson 1 provides a comprehensive review of binary (conversion, addition, overflow), hexadecimal (conversion, uses), character encoding (ASCII/Unicode — with emphasis on Unicode for Chinese), images (resolution, colour depth, file size calculation), sound (sampling rate, resolution), and compression (lossy vs lossless, RLE). Lesson 2 is dedicated to timed past paper practice with mark scheme review. This review ensures students who covered this content in Y9S1 have fresh, exam-ready understanding. The content represents approximately 20-25% of Paper 1 marks.

Syllabus Ref: CIE 0478 Topic 1.1, 1.2, 1.3, 1.4

Prerequisites:

- Y9S1: All Data Representation topics (assumed prior knowledge)
- This week: Revision and consolidation, not first teaching

LESSON 9.1 — COMPREHENSIVE DATA REPRESENTATION REVIEW

Topic: Binary, Hexadecimal, ASCII/Unicode, Images, Sound, Compression

Duration: 40 minutes

Lesson Type: Revision with Quick-Fire Recall and Worked Examples

Syllabus Ref: CIE 0478 Topic 1.1–1.4

Learning Objective:

By the end of this lesson, all students will be able to convert between binary, denary, and hexadecimal, explain ASCII and Unicode encoding,

calculate image and sound file sizes, describe lossy and lossless compression, and solve binary addition problems including overflow.

Key Vocabulary:

- binary = 二进制 (er jin zhi)
- denary/decimal = 十进制 (shi jin zhi)
- hexadecimal = 十六进制 (shi liu jin zhi)
- bit = 位 (wei)
- byte = 字节 (zi jie)
- ASCII = ASCII 码 (ASCII ma)
- Unicode = Unicode 码 (Unicode ma)
- resolution = 分辨率 (fen bian lv)
- colour depth = 颜色深度 (yan se shen du)
- sampling rate = 采样率 (cai yang lv)
- compression = 压缩 (ya suo)
- lossy = 有损 (you sun)
- lossless = 无损 (wu sun)
- overflow = 溢出 (yi chu)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Without a calculator — convert:
- | (a) 10110101 from binary to denary
- | (b) 205 from denary to binary"
- | Write working in notebook. This activates prior knowledge
- | and identifies who needs support.

3-12 min | DIRECT INSTRUCTION — Rapid Review

- | (Slide 1) Binary <-> Denary:
- | 128 64 32 16 8 4 2 1
- | 1 0 1 1 0 1 0 1
- | = $128 + 32 + 16 + 4 + 1 = 181$
- | Denary to binary: find largest power of 2, subtract,
- | repeat. Or divide by 2, record remainders.
- | (Slide 2) Binary Addition & Overflow:
- | 0111 (7)
- | + 0101 (5)
- | 1100 (12)

| Overflow: When the result exceeds what can be stored.

| 4-bit range: 0 to 15. $8 + 10 = 18 \rightarrow$ overflow!

| (Slide 3) Binary $\leftrightarrow$ Hexadecimal:

| Split binary into groups of 4 (from right):

| 1011 0101 $\rightarrow$ B 5 $\rightarrow$ 0xB5

| Hex uses: 0-9, A(10), B(11), C(12), D(13), E(14), F(15)

| Why hex? Compact representation of binary. Easier to

| read and write than long binary strings.

| (Slide 4) ASCII & Unicode:

| ASCII: 7 bits, 128 characters. English letters, numbers,

| symbols only. Cannot represent Chinese characters!

| Unicode: 16+ bits, over 100,000 characters. Includes

| Chinese, Arabic, emoji, mathematical symbols.

| Used by all modern systems.

| Why Unicode matters for us:

| "我们" in Unicode = U+6211 U+4EEC

| Your phone, computer, WeChat — all use Unicode

| (Slide 5) Images — File Size Calculation:

| File size = width x height x colour depth (in bits)

| Example: 800 x 600 pixels, 24-bit colour

| Size = $800 \times 600 \times 24 = 11,520,000$ bits

| In bytes: $/ 8 = 1,440,000$ bytes

| In KB: $/ 1024 = 1406.25$ KB

| In MB: $/ 1024 = 1.37$ MB

| Colour depth:

| 1 bit $\rightarrow$ 2 colours (black/white)

| 8 bit $\rightarrow$ 256 colours

| 24 bit $\rightarrow$ 16.7 million colours ("true colour")

| 32 bit $\rightarrow$ 16.7 million + transparency

| (Slide 6) Sound — File Size Calculation:

| File size = sampling rate x resolution x duration (seconds)

| Example: 44,100 Hz, 16-bit, 3 minutes

| Duration: $3 \times 60 = 180$ seconds

| Size = $44100 \times 16 \times 180 = 127,008,000$ bits

| In bytes: $/ 8 = 15,876,000$ bytes ~ 15.1 MB

| Higher sampling rate = better quality = larger file

- | (Slide 7) Compression:
- | LOSSY: Permanently removes data. Cannot recover original.
- | Used for: images (JPEG), sound (MP3), video (MP4)
- | Advantage: Much smaller files
- | Disadvantage: Quality reduced (artifacts, distortion)
- | LOSSLESS: Reduces size without losing data. Can recover original.
- | Used for: text (ZIP), some images (PNG), some sound (FLAC)
- | Method: RLE (Run-Length Encoding) — replaces repeated values with count + value
- | Example: AAAAAABBBCCCC -> 6A3B4C

12-25 min | GUIDED PRACTICE

- | Exercise 1 (whole class):
- | "Convert 1110 1100 from binary to hex."
- | -> 1110 = E, 1100 = C -> 0xEC
- | Exercise 2 (pairs):
- | "Calculate the file size of a 1024 x 768 image with 16-bit colour depth. Give answer in KB."
- | -> $1024 \times 768 \times 16 = 12,582,912$ bits
- | -> $/ 8 = 1,572,864$ bytes
- | -> $/ 1024 = 1536$ KB
- | Exercise 3 (individual):
- | "Explain why a photo of this slide saved as JPEG will be smaller than saved as BMP. What quality trade-off occurs?"
- | -> JPEG uses lossy compression. Smaller file but some image detail is permanently lost.
- | Exercise 4 (whole class):
- | "A 4-bit register holds 1111. We add 0001. What happens?"
- | -> $1111 + 0001 = 10000$ (which is 16 in denary)
- | -> But 4 bits can only hold 0-15!
- | -> Result stored: 0000 (overflow has occurred!)

25-35 min | INDEPENDENT PRACTICE

- | Worksheet 9A — Data Representation Review
- | Section A: Binary and hexadecimal conversions
- | Section B: ASCII/Unicode and character encoding
- | Section C: Image and sound file size calculations
- | Section D: Compression and overflow

| (Worksheet 9A distributed — see below)

35-40 min | PLENARY + HOMEWORK

| Plenary: "Data Representation Speed Round"

| "8 bits = how many bytes?" -> "1!"

| "1024 bytes = how many KB?" -> "1!"

| "Hex A = denary?" -> "10!"

| "Unicode vs ASCII — which supports Chinese?" -> "Unicode!"

| "JPEG — lossy or lossless?" -> "Lossy!"

| "PNG — lossy or lossless?" -> "Lossless!"

| Homework: Create a personal revision card for Data

| Representation. Include: conversion rules, formulas for

| file sizes, compression types, and one worked example for

| each topic.

EAL Support:

- Complete revision summary sheet (bilingual) with all formulas
- Conversion place-value chart (128 64 32 16 8 4 2 1) on every desk
- Hexadecimal digit reference (0-F with denary equivalents)
- File size calculation formula card
- Colour-coded: binary (blue), hex (green), denary (orange)
- Cognate: "digital" -> 数字, "code" -> 编码, "compress" -> 压缩

Differentiation:

Emerging | Use conversion tables and calculators. Focus on binary<->denary

| only. Complete Sections A (denary->binary only) and D only.

| Provided worked examples for all questions.

Developing | Complete all conversions and calculations independently.

| Show all working. Complete all sections.

Secure | Extension: Solve complex multi-step problems. Calculate

| the storage needed for 1 hour of 48kHz, 24-bit stereo

| audio. Explain why CDs use 44.1kHz sampling rate.

| Research: What is the Nyquist theorem and how does it

| relate to sampling?

Resources Needed:

- Slide deck (all revision topics with worked examples)
- Revision summary sheets (bilingual)
- Conversion place-value charts
- Worksheet 9A (one per student)

- Mini-whiteboards for speed round
- Calculators (optional for working checking)

LESSON 9.2 — PAST PAPER PRACTICE: DATA REPRESENTATION

Topic: Timed Exam Practice, Common Mistakes, Mark Scheme Review

Duration: 40 minutes

Lesson Type: Exam Practice with Review

Syllabus Ref: CIE 0478 Topic 1.1–1.4 (Past Paper Questions)

Learning Objective:

By the end of this lesson, all students will be able to answer IGCSE-style exam questions on Data Representation, identify common mistakes, apply mark scheme criteria to their answers, and improve exam technique.

Key Vocabulary:

- past paper = 历年试题 (li nian shi ti)
- timed practice = 计时练习 (ji shi lian xi)
- mark scheme = 评分标准 (ping fen biao zhun)
- exam technique = 考试技巧 (kao shi ji qiao)
- common mistake = 常见错误 (chang jian cuo wu)
- working = 计算过程 (ji suan guo cheng)

Lesson Structure (40 Minutes):

0-3 min | DO NOW

- | "Convert 0x2F to binary and to denary. Show all working."
- | Quick revision question to settle students.
- | (Answer: 0010 1111 binary = 47 denary)

3-12 min | DIRECT INSTRUCTION

- | (Slide 1) Exam technique for Data Representation:
 1. ALWAYS show your working — marks awarded for method
 2. State units clearly (bits, bytes, KB, MB)
 3. For file size: write the formula first, then substitute
 4. For conversions: show place values or division steps
 5. Check your answer is reasonable
 - (A photo cannot be 500 GB!)
- | (Slide 2) Common Mistakes — DO NOT:
 - | X Forget to divide by 8 when converting bits to bytes
 - | X Confuse colour depth with number of colours
 - (24-bit = 16.7 million colours, NOT 24 colours!)

- | X Use 1000 instead of 1024 for binary prefixes
- | X Forget overflow can occur in binary addition
- | X Say ASCII can represent Chinese characters
- | X Confuse lossy and lossless compression
- | (Slide 3) Mark scheme keywords:
 - | - "Explain" = give reason why
 - | - "Calculate" = show working + final answer
 - | - "State" = short answer, no explanation needed
 - | - "Describe" = give details in sequence
 - | - "Compare" = identify similarities AND differences
- | (Slide 4) Worked exam question:
 - | "A bitmap image has a resolution of 640 x 480 pixels
 - | and uses 256 colours.
 - | (i) State the colour depth required. [1]
 - | (ii) Calculate the file size in KB. [3]"
 - | Answer:
 - | (i) 8 bits [1] — because $2^8 = 256$
 - | (ii) File size = $640 \times 480 \times 8 = 2,457,600$ bits [1]
 - | = $2,457,600 / 8 = 307,200$ bytes [1]
 - | = $307,200 / 1024 = 300$ KB [1]

12-25 min | TIMED EXAM PRACTICE (15 minutes)

- | Students complete Section A of Worksheet 9B in exam
- | conditions: silence, no talking, no calculators for
- | simple conversions.
- | Teacher circulates, ensures focus.

25-35 min | MARK SCHEME REVIEW

- | Teacher displays mark scheme on board.
- | Students self-mark with green pen.
- | Discussion of each question:
 - | - What was the examiner looking for?
 - | - Where did students lose marks?
 - | - What working was required?
- | Common errors collected on board.

35-40 min | PLENARY + HOMEWORK

- | Plenary: "One thing I learned today" — each student says
- | one thing they will remember for the exam.

| Homework: Complete Section B of Worksheet 9B. Self-mark
| using answer key. Write a list of 5 personal "do not
| forget" points for Data Representation.

EAL Support:

- Exam vocabulary reference card (bilingual)
- Sentence frames for explain/compare questions:
"The answer is ____ because ____"
"____ is different from ____ because ____"
- Mark scheme explained in simple language
- Common mistakes poster on wall
- Highlight that "show working" means write every step

Differentiation:

Emerging | Extra time for exam practice. Simplified questions with
| fewer marks. Provided formula sheet. Complete Section A
| only with teacher support.

Developing | Complete timed practice independently. Self-mark with
| mark scheme. Complete all sections. Identify own weak
| areas for revision.

Secure | Extension: Complete additional past paper questions
| under timed conditions. Peer-mark using official mark
| scheme. Write model answers for 4-mark "explain"
| questions. Create a revision guide for a peer.

Resources Needed:

- Slide deck (exam technique, common mistakes, worked questions)
- Worksheet 9B (one per student)
- Mark scheme printed for self-marking
- Green pens for self-marking
- Timer for exam practice
- Formula sheet (optional, for emerging)

WORKSHEET 9A — DATA REPRESENTATION REVIEW

Name: _____ Class: _____ Date: _____

Learning Objective: I can convert between number systems, calculate file sizes, and explain data representation concepts.

Bilingual Support:

- binary = 二进制 (er jin zhi)

- denary = 十进制 (shi jin zhi)
- hexadecimal = 十六进制 (shi liu jin zhi)
- ASCII = ASCII 码
- Unicode = Unicode 码
- resolution = 分辨率 (fen bian lv)
- colour depth = 颜色深度 (yan se shen du)
- compression = 压缩 (ya suo)

SECTION A: Binary and Hexadecimal [10 marks]

135. Convert 1010 1101 from binary to denary. Show working. [2]

Answer: _____

2. Convert 237 from denary to binary. Show working. [2]

Answer: _____

3. Convert 1100 1110 from binary to hexadecimal. [2]

Answer: _____

4. Convert 0x3A from hexadecimal to binary. [2]

Answer: _____

5. Calculate: 0110 (6)

+ 0101 (5)

Show working and state the denary answer. [2]

Answer: _____

SECTION B: ASCII and Unicode [4 marks]

136. Explain why ASCII is not suitable for representing Chinese text. [2]

7. State TWO advantages of Unicode over ASCII. [2]

SECTION C: Images and Sound [8 marks]

137. A digital photo has resolution 1920 x 1080 and uses 24-bit colour.

(a) Calculate the file size in bytes. Show working. [2]

Answer: _____ bytes

(b) Convert your answer to megabytes (MB). [2]

Answer: _____ MB

138. A sound file is recorded at 22,050 Hz with 8-bit resolution for 2 minutes (mono).

(a) Calculate the file size in bytes. Show working. [2]

Answer: _____ bytes

(b) Explain why increasing the sampling rate improves sound quality. [2]

SECTION D: Compression and Overflow [8 marks]

139. Explain the difference between lossy and lossless compression.

For each, give one example file format. [4]

140. Apply Run-Length Encoding (RLE) to compress this image data:

A A A B B C C C C D D D D D

Write your compressed data: _____ [2]

141. An 8-bit register holds the binary value 1111 1111 (255).

One is added to this value.

(a) What is the mathematical result? _____ [1]

(b) What value is actually stored in the 8-bit register? _____ [1]

(c) What has occurred? _____ [1]

WORKSHEET 9B — PAST PAPER PRACTICE: DATA REPRESENTATION

Name: _____ Class: _____ Date: _____

Learning Objective: I can answer IGCSE exam-style questions on Data Representation under timed conditions.

Bilingual Support: See Worksheet 9A for vocabulary.

TIME ALLOWED: 20 minutes for Section A (exam conditions)

SECTION A: Timed Exam Questions [20 marks]

142. (a) Convert the denary number 93 to binary. [2]

(b) Convert the binary number 1101 0110 to denary. [2]

(c) Convert the binary number 1110 1001 to hexadecimal. [2]

143. An image has a width of 800 pixels, a height of 600 pixels, and a colour depth of 16 bits.

(a) Calculate the file size of the image in bytes. [2]

(b) The image is saved using lossless compression and the compressed file size is 480 KB.

Calculate the compression ratio (original : compressed). [2]

3. Explain why Unicode is used instead of ASCII in modern

computer systems. [3]

4. A sound recording is made with these settings:

- Sampling rate: 44,100 Hz
- Resolution: 16 bits
- Duration: 5 seconds
- Channels: 2 (stereo)

(a) Calculate the file size in bytes. Show working. [3]

(b) Explain what would happen to the file size and quality if the sampling rate was halved. [2]

144. (a) Explain the difference between lossy and lossless compression. [2]

(b) Give one situation where lossy compression would NOT be suitable. Explain your answer. [2]

ANSWER KEY — WEEK 9

WORKSHEET 9A:

SECTION A:

145. $1010\ 1101 = 128+32+8+4+1 = 173$ [2]

(1 mark for correct method showing place values, 1 mark for answer)

146. $237 = 1110\ 1101$ [2]

(1 mark for method e.g., subtraction of powers of 2 or division, 1 mark for correct answer)

Method: $237-128=109$, $109-64=45$, $45-32=13$, $13-8=5$, $5-4=1$, $1-1=0$

-> 1110 1101

147. $1100 = C$, $1110 = E$ -> 0xCE [2]

(1 mark for splitting into nibbles, 1 mark for correct hex)

148. $0x3 = 0011$, $0xA = 1010$ -> 0011 1010 [2]

(1 mark for each nibble, 1 mark for full correct answer)

149. 0111

+0101

1011 (11 in denary) [2]

(1 mark for correct binary addition, 1 mark for denary answer)

SECTION B:

150. ASCII only has 128 characters (7 bits) [1]. It does not include Chinese characters or any non-English characters [1].

7. Any two valid advantages:

- Unicode can represent many more characters (over 100,000) [1]
- Unicode includes Chinese/Arabic/other languages [1]
- Unicode is the international standard [1]
- Backward compatible with ASCII [1]

(Award 1 mark per valid advantage, max 2)

SECTION C:

151. (a) $1920 \times 1080 \times 24 = 49,766,400$ bits [1]

$49,766,400 / 8 = 6,220,800$ bytes [1]

(b) $6,220,800 / 1024 = 6075$ KB [1]

$6075 / 1024 = 5.93$ MB (accept 6.07 MB if used 1000) [1]

(Accept either 1024 or 1000 conversion if consistent)

152. (a) $22050 \times 8 \times 120 = 21,168,000$ bits [1]

$21,168,000 / 8 = 2,646,000$ bytes [1]

(b) Higher sampling rate captures more detail of the sound wave [1].

More samples per second means the digital representation is

closer to the original analogue sound [1].

SECTION D:

153. Lossy compression permanently removes data from the file [1].

The original file cannot be perfectly recovered [1].

Example: JPEG, MP3, MP4 [1] (any one)

Lossless compression reduces file size without losing any data [1].

The original file can be perfectly recovered [1].

Example: PNG, ZIP, FLAC [1] (any one)

(Max 4 marks — need definitions and one example of each)

154. 4A2B5C6D [2]

(1 mark for correct counts, 1 mark for correct sequence)

155. (a) 256 [1]

(b) 0000 0000 [1] (or 0)

(c) Overflow [1] has occurred.

WORKSHEET 9B:

SECTION A: (Mark allocation as indicated in questions)

156. (a) $93 = 64 + 16 + 8 + 4 + 1 = 0101\ 1101$ [2]

(1 method, 1 answer)

(b) $1101\ 0110 = 128 + 64 + 16 + 4 + 2 = 214$ [2]

(1 method, 1 answer)

(c) $1110 = E$, $1001 = 9 \rightarrow 0xE9$ [2]

(1 for splitting, 1 for answer)

157. (a) $800 \times 600 \times 16 = 7,680,000$ bits [1]

$7,680,000 / 8 = 960,000$ bytes [1]

(b) Original: 960,000 bytes = 937.5 KB [1]

Ratio: $937.5 : 480 = 1.95 : 1$ (accept approximately 2:1) [1]

(Accept if student calculates ratio differently but correctly)

158. Unicode supports many more characters than ASCII [1]. Unicode includes characters from many different languages (not just English) such as Chinese, Arabic, etc. [1]. Modern systems need to support global communication in many languages [1].

(Accept: Unicode is backwards compatible with ASCII)

159. (a) File size = $44100 \times 16 \times 5 \times 2 = 7,056,000$ bits [1]

$7,056,000 / 8 = 882,000$ bytes [2]

(1 mark for formula with all values, 1 for calculation,
1 for final answer in bytes)

Note: Must include $\times 2$ for stereo channels

(b) File size would be halved/smaller [1].

Quality would be reduced — some high frequencies would be lost / less accurate representation of the sound [1].

160. (a) Lossy: permanently removes data, original cannot be recovered [1]

Lossless: no data lost, original can be recovered [1]

(b) Any valid situation: storing medical records / text documents / financial data / legal documents where every detail matters [1].

Explanation: because losing any data could be dangerous/unacceptable/illegal [1]. Accept any reasonable answer.

END OF YEAR 10 SEMESTER 1 — WEEKS 1-9

COURSE MAP: WHAT'S NEXT?

Year 10 Semester 2 (Weeks 10-18) will cover:

- Week 10: Paper 1 Review — Hardware & Systems Architecture
- Week 11: Paper 1 Review — Software & Operating Systems
- Week 12: Paper 1 Review — Internet & Security
- Week 13: Paper 1 Review — Emerging Technology
- Week 14: Paper 2 Review — Consolidated Programming Practice
- Week 15: Paper 2 Review — Advanced Algorithms
- Week 16: Mock Paper 1 Practice
- Week 17: Mock Paper 2 Practice
- Week 18: End-of-Year Assessment & Gap Analysis

ASSESSMENT ALIGNMENT SUMMARY — SEMESTER 1

Topic 9 — Databases (Weeks 1-3):

- 9.1 Flat-file vs relational databases OK
- 9.2 Tables, records, fields, primary keys OK
- 9.3 Entity-Relationship Diagrams OK
- 9.4 Relationships (1:1, 1:M, M:N) OK
- 9.5 SQL: SELECT, FROM, WHERE, AND, OR OK
- 9.6 SQL: ORDER BY, wildcards, BETWEEN, IN OK
- 9.7 Foreign keys, linking tables OK
- 9.8 INNER JOIN OK
- 9.9 Normalisation (1NF, 2NF, 3NF) OK

Topic 10 — Boolean Logic (Weeks 4-5):

- 10.1 AND, OR, NOT gates and truth tables OK
- 10.2 Logic expressions and circuit diagrams OK
- 10.3 NAND, NOR, XOR gates OK
- 10.4 Truth tables with 3 inputs OK
- 10.5 Half-adders OK
- 10.6 NAND as universal gate OK

Topic 7 — Bubble Sort (Week 6):

- 7.1 Bubble sort algorithm OK
- 7.2 Pseudocode and Python implementation OK
- 7.3 Efficiency analysis ($O(n^2)$) OK

Topic 8 — Validation (Week 7):

- 8.1 Range, type, length checks OK
- 8.2 Presence, format checks OK
- 8.3 Combining validation checks OK
- 8.4 Robust input routines OK

Topic 3.4 — Network Hardware (Week 8):

- 3.4.1 NIC, MAC address, IP address (IPv4/IPv6) OK
- 3.4.2 Routers, switches OK
- 3.4.3 Network topologies (star, bus, mesh) OK
- 3.4.4 Client-server vs peer-to-peer OK

Topic 1 — Data Representation Review (Week 9):

- 1.1 Binary, hex, conversions, addition, overflow OK
- 1.2 Images (resolution, colour depth, file size) OK
- 1.3 Sound (sampling rate, resolution, file size) OK
- 1.4 Compression (lossy, lossless, RLE) OK

DIFFERENTIATION & EAL SUMMARY

EAL Support Strategies Used Throughout:

- OK Bilingual keyword lists (English / Chinese / Pinyin) in every lesson
- OK Visual anchors and colour-coded diagrams
- OK Sentence frames for explanations
- OK Think-Pair-Share before whole-class responses
- OK Cognates highlighted
- OK Word banks on every worksheet
- OK Wait time increased to 5+ seconds

Differentiation Applied:

- OK Emerging: scaffolding, sentence starters, partially completed tasks
- OK Developing: standard tasks, independent completion
- OK Secure: extension challenges, optimisation problems, research tasks

TOTALS FOR THIS DOCUMENT

- Weeks: 9
- Lessons: 18 (2 per week x 9 weeks)
- Worksheets: 18 (2 per week x 9 weeks)
- Answer Keys: 18 (1 per week x 9 weeks)
- Topics: 6 major syllabus areas
- Syllabus Ref: CIE IGCSE Computer Science 0478

END OF DOCUMENT